Jet Ranger Arduino and C# Program Summary

Master document for the Bell 206 Jet Ranger cockpit: Arduino sketches, Windows programs and PCBs

Regenerated 18 October 2026

Contents

1 Project overview....5
1.1 How it fits together....5
1.2 Arduino sketches....6
1.3 Windows programs....7
1.4 PCBs....7
1.5 Where things stand....8
2 Network and communication....9
2.1 Devices and addresses....9
2.2 UDP ports....9
2.3 Packet format....10
2.4 Board life cycle....11
2.5 Power rules in the bridges....12
2.6 Remote reset....13
2.7 Debug and monitoring....14
3 Radio controller (172.16.1.101)....15
3.1 Program flow....15
3.2 Pin usage....16
3.3 Data from the PC (UDP 13136)....16
3.4 Commands to the PC (UDP 27001)....17
3.5 Tuning steps....17
3.6 Power and blanking....17
3.7 Diagnostics....17
3.8 Known issues....18
4 Servo controller (172.16.1.102)....19
4.1 Program flow....19
4.2 Servos and calibration....20
4.3 Warning lamps....21
4.4 Debounced switch inputs....21
4.5 On-demand commands....22
4.6 Pin usage....22
4.7 Known issues....22
5 Upper controller (172.16.1.103)....23
5.1 Program flow....23
5.2 Pin usage....23
5.3 What it sends....24
5.4 Remote reset....24
5.5 Known issues....24
6 USB HID board (172.16.1.11)....25
6.1 Program flow....25
6.2 How a switch becomes a joystick button....26
6.3 Switch mapping in use....26

6.4 Pin usage and network....26
6.5 Known issues....27
7 Stepper controller (172.16.1.105)....28
7.1 Gauges and calibration....28
7.2 Program flow....29
7.3 Packets....29
7.4 On-demand swings....29
7.5 Pin usage....29
7.6 Network....30
7.7 Known issues....30
8 Dual stepper controller (172.16.1.106)....31
8.1 Gauges and calibration....31
8.2 OLED displays....32
8.3 Program flow....32
8.4 Packets....33
8.5 On-demand commands....33
8.6 Pin usage....33
8.7 Network....33
8.8 Known issues....33
9 OLED controller (172.16.1.104)....34
9.1 What it drives....34
9.2 Program flow....34
9.3 UDP interface....34
9.4 Pin usage....34
9.5 Network....35
9.6 Notes....35
10 FSUIPC bridge (FSUIPCWinformsAutoCS)....36
10.1 Program flow....36
10.2 FSUIPC offsets....37
10.3 Packets sent....37
10.4 Commands from the Radio controller (UDP 27001)....37
10.5 Notes and quirks....38
11 MSFS bridge (MSFSWinformsAutoCS)....39
11.1 Files....39
11.2 Program flow....40
11.3 Packets....40
11.4 SimVars requested....41
11.5 Commands from the panel (UDP 27001)....41
11.6 Aircraft profiles....42
11.6.1 FlyInside B206....42
11.6.2 CowaSim Jet Ranger....43
11.7 Warning lamps....43
11.8 Known gaps....44

12 Health Monitor (JetRangerHealthMonitor)....45
12.1 Program flow....45
13 Radio and Upper controller emulators....46
13.1 JetRangerRadioControllerEmulator....46
13.2 JetRangerUpperControllerEmulator....46
14 StepperVSITester....47
14.1 What it sends....47
15 SwingTester....48
15.1 How it works....48
15.2 Window....48
15.3 Warning lamps....49
15.4 Keep-alive....49
16 Debug listener (UDP reflector)....50
17 PCB notes....51
17.1 Boards in use....51
17.2 Warning Mute, Fuel Valve and TOT Clear inputs....52
17.3 Pin and copper check (12 October 2026)....52
17.4 KiCad: changing the revision number....52
Appendix A: UDP token reference....53
17.5 Gauge tokens....53
17.6 Display and status tokens....53
17.7 Lamp tokens (Servo .102)....53
17.8 Control tokens (C,<COMMAND>:<value>)....53
17.9 Messages from the Radio controller to the bridge (UDP 27001)....54
Appendix B: Building and flashing....55
17.10 Libraries....55
17.11 Compile and upload....55
17.12 COM port to board mapping....56
17.13 Sketch settings to know about....56
Appendix C: Calibration....57
Appendix D: How this document is built....58

1 Project overview

A home-built Bell 206 Jet Ranger cockpit: real instrument gauges, warning lamps, radio and overhead switch panels and small displays, driven from a flight simulator by Arduino Mega boards on a private Ethernet network. The simulator can be Microsoft Flight Simulator 2020 or 2024 (through SimConnect) or Prepar3D / FSX (through FSUIPC).

This document ties together the three parts of the repository: the **Arduino sketches**, the **Windows (C#) programs** and the **KiCad PCBs**. It is rebuilt from each project's own `PROGRAM_SUMMARY.md` and from the code, and it covers only the projects that are in use.

1.1 How it fits together

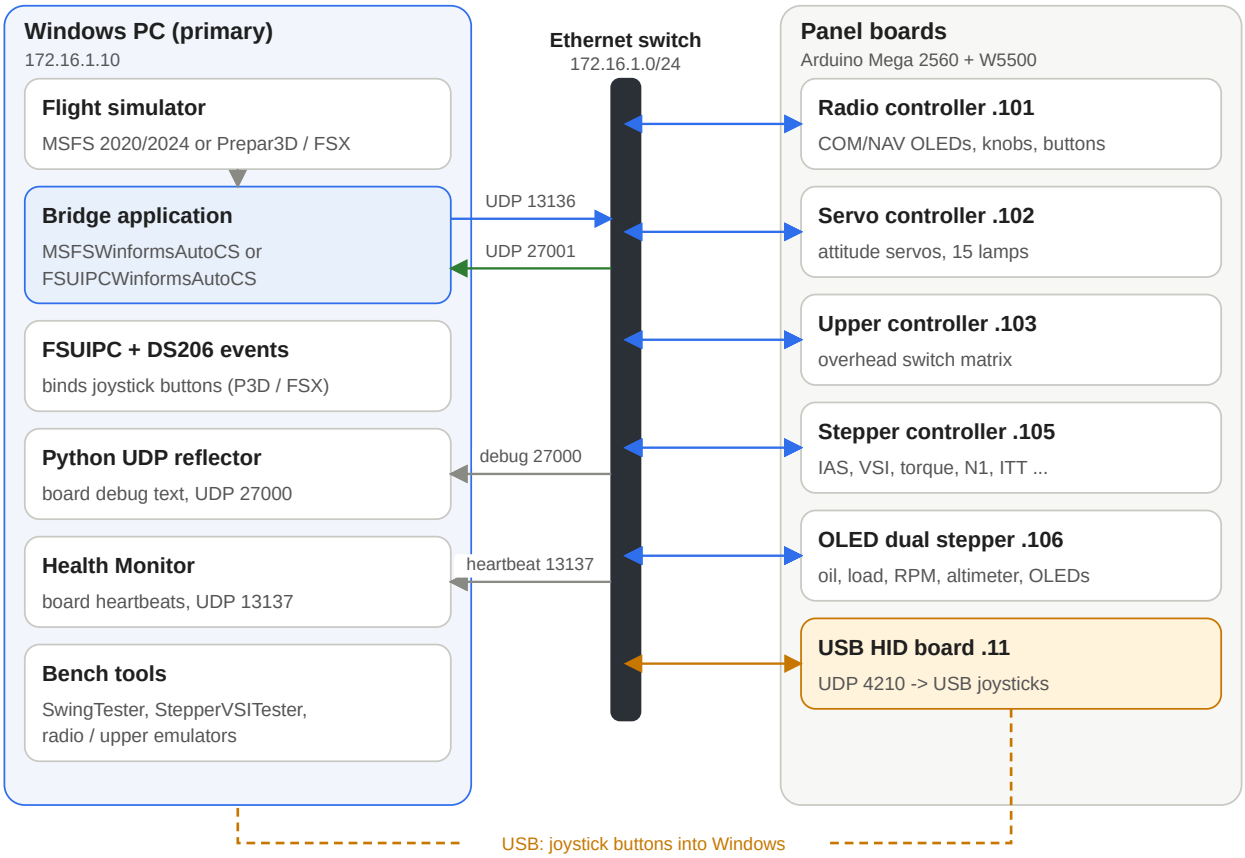


Figure 1 — The whole system: simulator, PC bridge, Ethernet and the panel boards

- The **simulator** supplies the flight data.
- A **bridge program** on the PC reads the data, converts it to real units and sends one short UDP text packet to each board about ten times a second.
- Each **Arduino board** turns the real units into stepper steps, servo positions, lamp states or display text with calibration tables in its own sketch.
- Switches and knobs go back the other way: the radio board sends commands to the bridge (UDP 27001), and the switch boards send joystick button changes to a USB HID board (UDP 4210) which Windows sees as game controllers.
- Every board sends a **heartbeat** every 10 seconds to the Health Monitor (UDP 13137) and its **debug text** to a listener on the PC (UDP 27000).

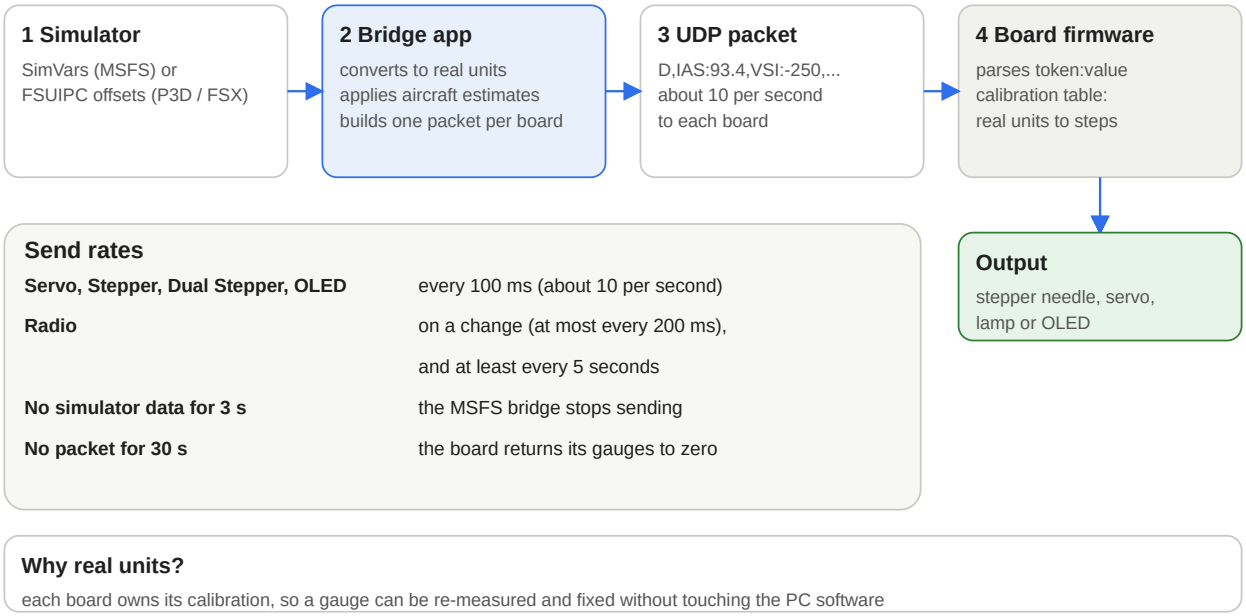


Figure 2 — From simulator value to moving needle

1.2 Arduino sketches

All sketches are in `Jet Ranger Arduino Sketches/`. Each is an Arduino Mega 2560 with a W5500 Ethernet module unless noted.

Sketch	IP	Role
`JET_RANGER_RADIO_CONTROLLER`	.101	COM and NAV frequency displays, eight tuning knobs, four frequency-swap switches; sends tuning and swap commands to the PC
`JET_RANGER_SERVO_CONTROLLER`	.102	Attitude indicator pitch and bank servos, 15 warning lamps, three front-panel switches (TOT clear, warning mute, fuel valve)
`JET_RANGER_UPPER_CONTROLLER`	.103	Overhead switch matrix (176 positions) and keypad; forwards changes to the USB HID board
`UDP_TO_HID`	.11	Leonardo-class board: UDP in, four 32-button joysticks and a keyboard out over USB
`JET_RANGER_OLED_CONTROLLER`	.104	Altitude and barometric-setting OLEDs; reads `ALT` and `BARO` from the bridge packet (the sketch also contains DCS-BIOS code from an A-10C console)
`JET_RANGER_STEPPER_CONTROLLER`	.105	Stepper gauges: airspeed, vertical speed, radar altitude, torque, gas producer, turbine out temperature, fuel quantity
`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER`	.106	Stepper gauges: engine and transmission oil, engine and rotor RPM, electrical load, fuel load, altimeter; four OLEDs (barometer, altimeter, clock, temperature)

Common to the three gauge boards (Servo, Stepper, Dual Stepper):

- bench-measured calibration tables with linear interpolation from real units to steps or servo positions, plus a raw bypass (`<CODE>RAW`) for calibration work;
- start-up self-test swings, also callable on demand (`C,SWINGTQ:1` and so on);
- a 30 second no-data watchdog that returns every gauge to zero;
- remote reset (`C,RESET:1`).

1.3 Windows programs

All programs are in `Jet Ranger/ C# Code/` (note the leading space in the folder name).

Program	Purpose
`FSUIPCWinformsAutoCS`	Bridge for Prepar3D / FSX using FSUIPC offsets
`MSFSWinformsAutoCS`	Bridge for MSFS 2020 / 2024 using SimConnect, with estimates for aircraft whose dials do not read the standard variables (FlyInside B206, CowaSim Jet Ranger)
`JetRangerHealthMonitor`	Traffic-light dashboard for the 10 second board heartbeats
`StepperVSITester`	Bench tester that sends real and raw values to the stepper boards and the servo board
`SwingTester`	Runs every gauge's swing individually or in a fixed order, lights the OLEDs and warning lamps, and resets boards
`JetRangerRadioControllerEmulator`, `JetRangerUpperControllerEmulator`	Stand-ins for the radio and upper boards when they are disconnected

Python helpers in `Jet Ranger Python Code/`: a SimConnect exporter, a UDP test receiver and a wake-on-LAN script. The UDP debug listener (port 27000) is a small Python reflector that prints every board's debug text and can forward a copy to a second listener.

Run **only one bridge at a time**: `FSUIPCWinformsAutoCS` and `MSFSWinformsAutoCS` both listen on UDP 27001.

1.4 PCBs

KiCad projects are in `Jet Ranger/PCBs/`. The boards used by the sketches above are:

- **Radio:** Jet Ranger Radio Auto Pilot and Jet Ranger COM and NAV
- **Servo:** Jet Ranger Front Panel Servo
- **Upper controller:** Jet Ranger Upper Controller
- **Steppers:** Jet Ranger 13 Stepper, Jet Ranger OLED Dual Stepper and one Jet Ranger X27 Small Stepper per gauge
- **Other panels and gauge boards:** Annunciator, Altimeter, Audio Panel, Auto Pilot Panel, ADF Panel and ADF Controller, and the Simkits gauge boards

The PCB notes chapter records what was verified from the KiCad files.

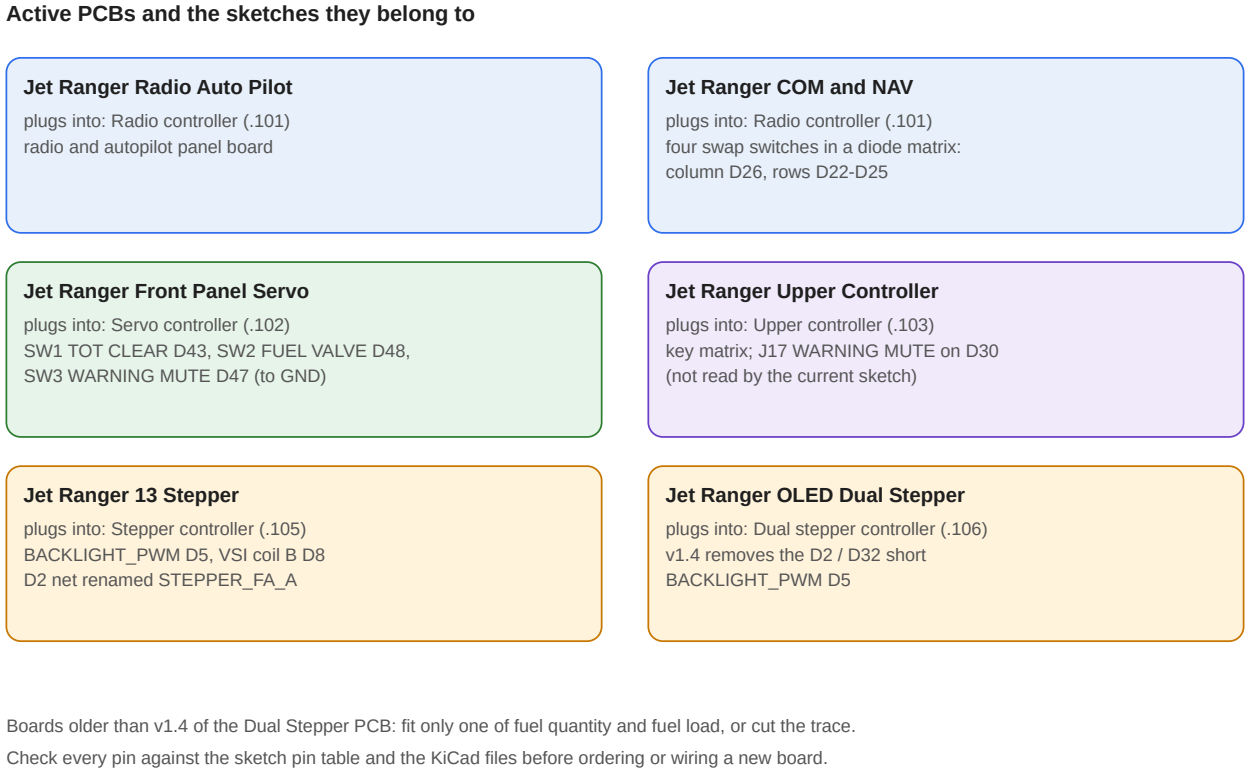


Figure 3 — The active PCBs and the sketches they belong to

1.5 Where things stand

Bench-verified with UDP debug logs from the boards and real swing timings: every swing command, the OLED lamp test, the temperature OLED, the keep-alive, the debounced servo inputs, the radio matrix, the remote reset on every board except the Upper controller, and the full SwingTester sequence. The FSUIPC bridge has been run against Prepar3D v6.1 and the MSFS bridge against MSFS 2024 with the FlyInside B206 and the CowaSim Jet Ranger.

Open items:

- The three servo-board switches reach the HID board but are not sent to the PC itself (the bridges treat an unrecognised string on port 27001 as a radio frequency, so they would need updating first).
- The Upper controller PCB has a Warning Mute connector on D30, but the Upper sketch scans analog pins A9 to A15 for its keypad only, so it may not read it.
- The Upper controller's remote reset has not been bench-tested; the board was not connected.
- The estimates in the MSFS bridge for the FlyInside and CowaSim aircraft come from a few dial readings; check them at other power settings.
- The Radio controller emulator still sends a bare frequency for every radio, so only COM1 tunes from it; the real board sends labelled COM2, NAV1 and NAV2 values, which both bridges understand.
- The Dual Stepper's RAM use is high (75 to 77 percent); be careful adding code to it.
- Calibration tables, swing times and waits come from bench measurements; recheck them after any mechanical change to a gauge.

2 Network and communication

2.1 Devices and addresses

All devices sit on one wired `172.16.1.0/24` network with fixed addresses; there is no DHCP. The firmware sends its debug and heartbeat traffic to `172.16.1.10`, so the primary PC must hold that address (or `reflectorIP` must be changed in each sketch and the sketch uploaded again).

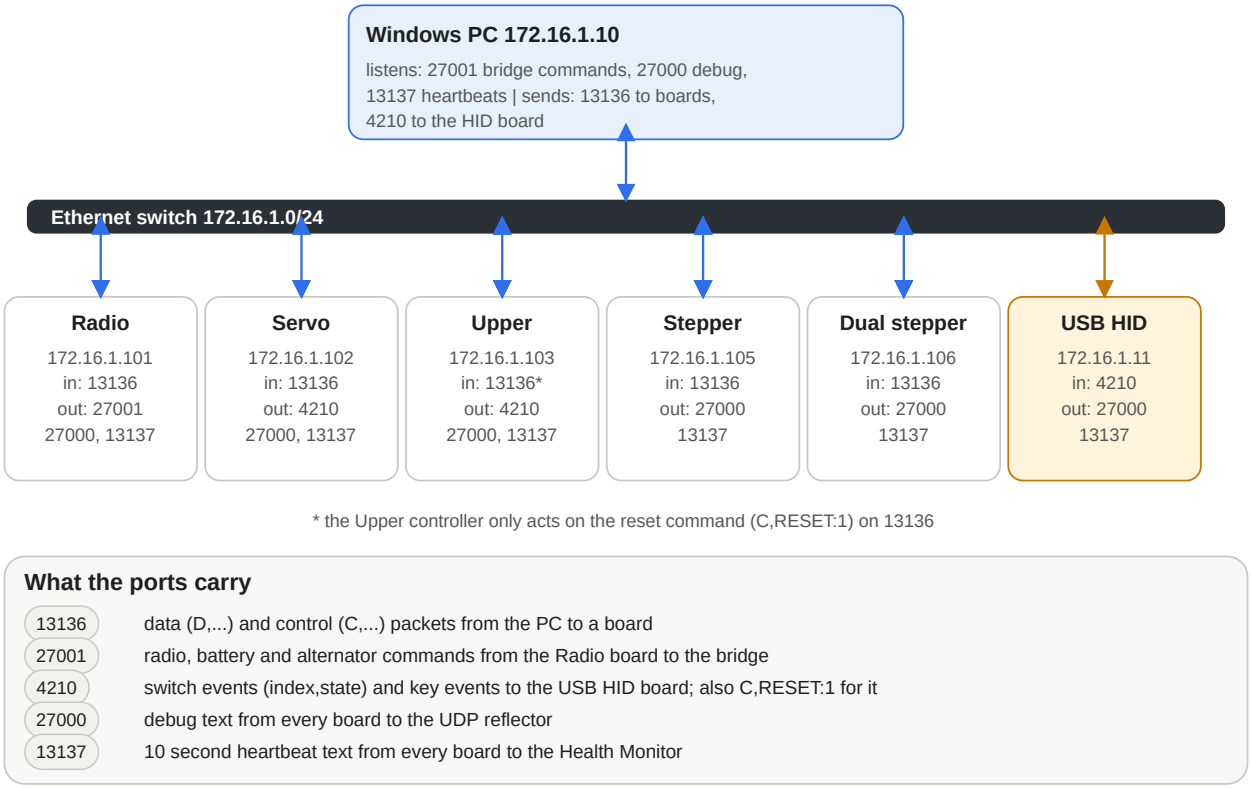


Figure 4 — Who talks to whom

Device	IP	MAC	Heartbeat text
Primary PC (simulator, bridge, listeners)	172.16.1.10		
USB HID board	172.16.1.11	DE:AD:BE:EF:FE:ED	`JOYSTICK`
Radio controller	172.16.1.101	A8:61:0A:9E:83:01	`COMM_NAV`
Servo controller	172.16.1.102	A8:61:0A:9E:83:02	`SERVO`
Upper controller	172.16.1.103	A8:61:0A:9E:83:03	`UPPER_INPUT`
OLED controller	172.16.1.104	A8:61:0A:67:83:68	none
Stepper controller	172.16.1.105	A8:61:0A:67:83:69	`STEPPER`
Dual stepper controller	172.16.1.106	A8:61:0A:67:83:6A	`DUAL_STEPPER`

Every live board has its own IP and MAC address.

2.2 UDP ports

Port	Direction	Purpose
13136	PC to boards	Data (`D,`) and control (`C,`) packets
27001	Radio controller to PC bridge	Standby frequencies and commands such as `COM1_RADIO_SWAP`
4210	Servo and Upper controllers to USB HID board	Button changes (`index,state`), keys (`KEY,key,state`), reset
13137	Boards to Health Monitor	Heartbeat text every 10 seconds

Port	Direction	Purpose
27000	Boards to debug listener	Debug text, each line prefixed with the board name
7788	Local on the boards	Source port opened at start-up for outgoing packets

2.3 Packet format

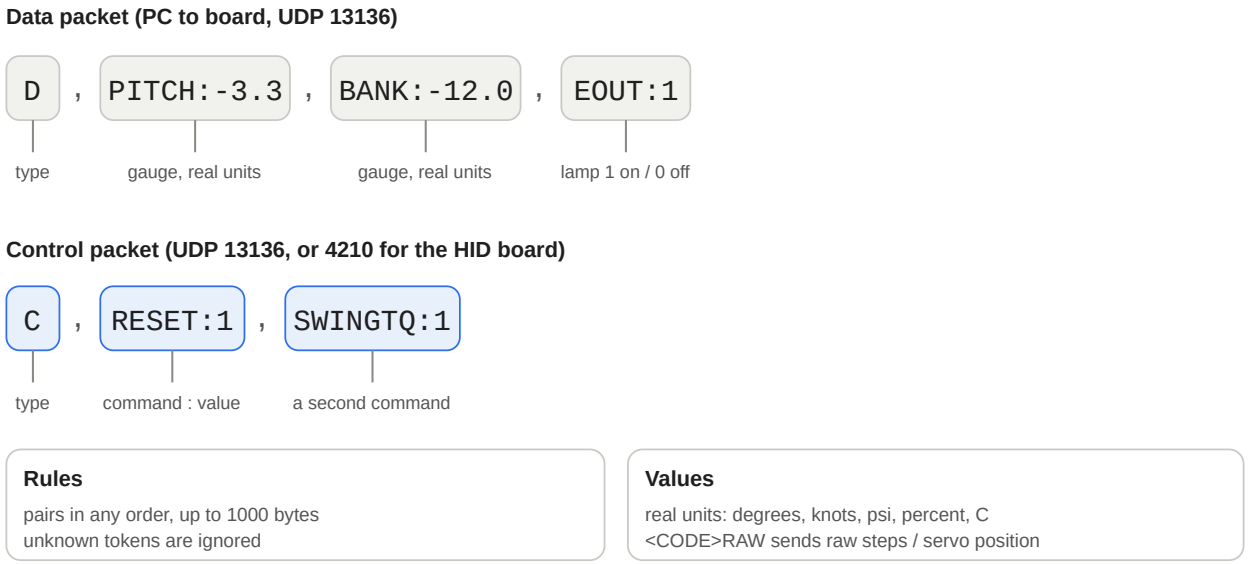


Figure 5 — Data and control packets

A packet is plain ASCII text, up to 1000 bytes.

- `D,CODE:value,CODE:value,...` is a **data** packet. It can carry any number of pairs, in any order. Unknown codes are ignored, so the PC can send only the values that changed.
- `C,COMMAND:value` is a **control** packet (swing tests, lamp test, keep-alive, reset). The value is usually ignored.
- Gauge codes take **real units**: degrees, knots, feet per minute, psi, percent, degrees C, US gallons. The board converts them with its own calibration table.
- A calibrated code usually has a raw twin with `RAW` on the end (for example `IASRAW:300`). It moves the needle to a raw step or servo position and skips the table, which is how new calibration points are measured.
- Lamp codes take `0` (off) or `1` (on).

Example sent to the servo board: `D,PITCH:5,BANK:-10,LOWF:0` sets 5 degrees pitch, minus 10 degrees bank and switches the Low Fuel lamp off.

To test without the simulator, send a packet from any PC on the subnet:

```
$u = New-Object System.Net.Sockets.UdpClient
$b = [Text.Encoding]::ASCII.GetBytes('D,PITCH:5,RLOW:1')
$u.Send($b, $b.Length, '172.16.1.102', 13136)
```

2.4 Board life cycle

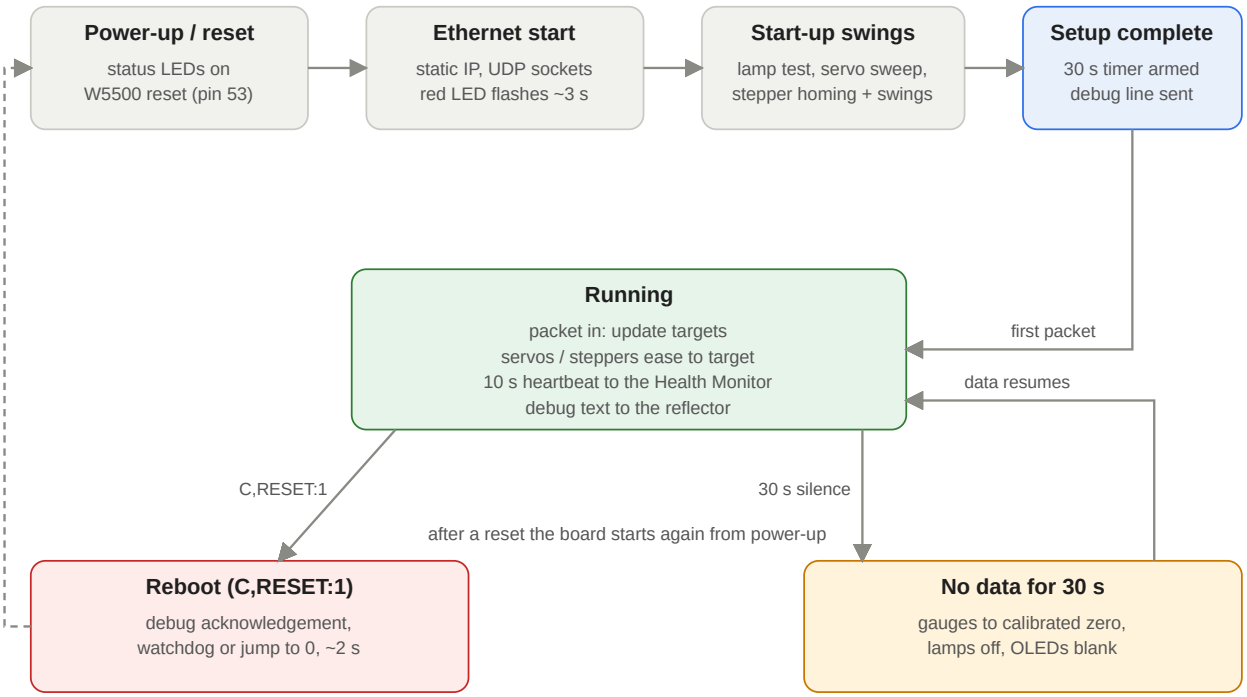


Figure 6 — What every gauge board does from power-up to shutdown

1. **Start-up:** status LEDs flash, the Ethernet module is reset, the board gets its static address and opens its sockets; the red LED flashes for a few seconds while the link settles.
2. **Self-test:** the gauge boards sweep each gauge (the Servo board also lights all lamps) and park at the calibrated zero.
3. **Running:** every 5 ms the board checks UDP 13136, stores each value as a new target and moves the needle towards it with smooth acceleration.
4. **Heartbeat:** every 10 seconds the board sends its heartbeat text to the Health Monitor.
5. **No-data watchdog:** if no packet (data or control) arrives for 30 seconds, every gauge returns to its calibrated zero. The Servo board also switches its lamps off, the Dual Stepper blanks its OLEDs and the Radio blanks its displays. The timer starts at the end of `setup()` so that a long self-test cannot use it up. Normal operation resumes with the next packet.

All sketches call `Ethernet.setRetransmissionTimeout(10)` after `Ethernet.begin()`. The W5500 default of 200 ms times 8 retries can block a UDP send for 1.6 seconds if the destination does not answer ARP (for example the PC is off); with 10 ms the worst case is about 80 ms, so one dead destination cannot stall gauge movement or button scanning.

2.5 Power rules in the bridges

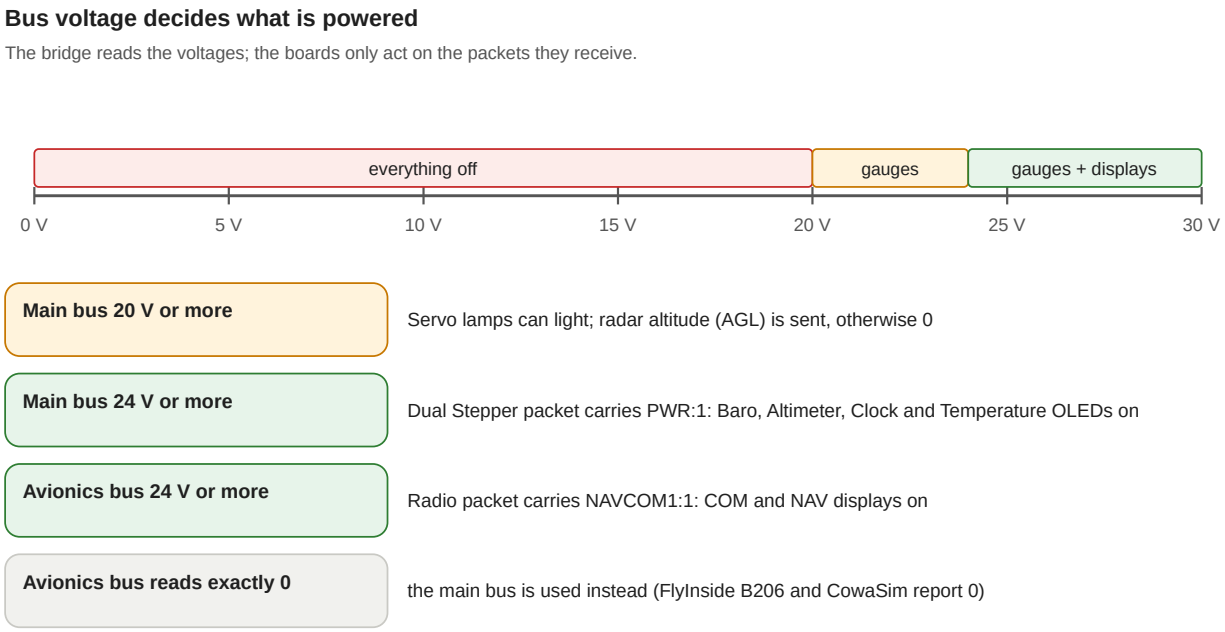


Figure 7 — How bus voltages switch the panel

- **Main bus below 20 V:** the Servo board's lamps are sent as off and the Stepper's radar altitude (`AGL`) is sent as zero.
- **Main bus below 24 V:** the bridge sends `PWR:0` to the Dual Stepper, which blanks its four OLEDs; `PWR:1` turns them back on.
- **Avionics bus below 24 V:** the bridge sends `NAVCOM1:0` to the Radio, which blanks its COM and NAV displays together. If the aircraft reports the avionics bus as exactly 0 (the FlyInside B206 does) the main bus is used instead.

There is no hysteresis on these thresholds. The 24 V figure is carried over from the FSUIPC bridge and has not been verified for every MSFS aircraft.

2.6 Remote reset

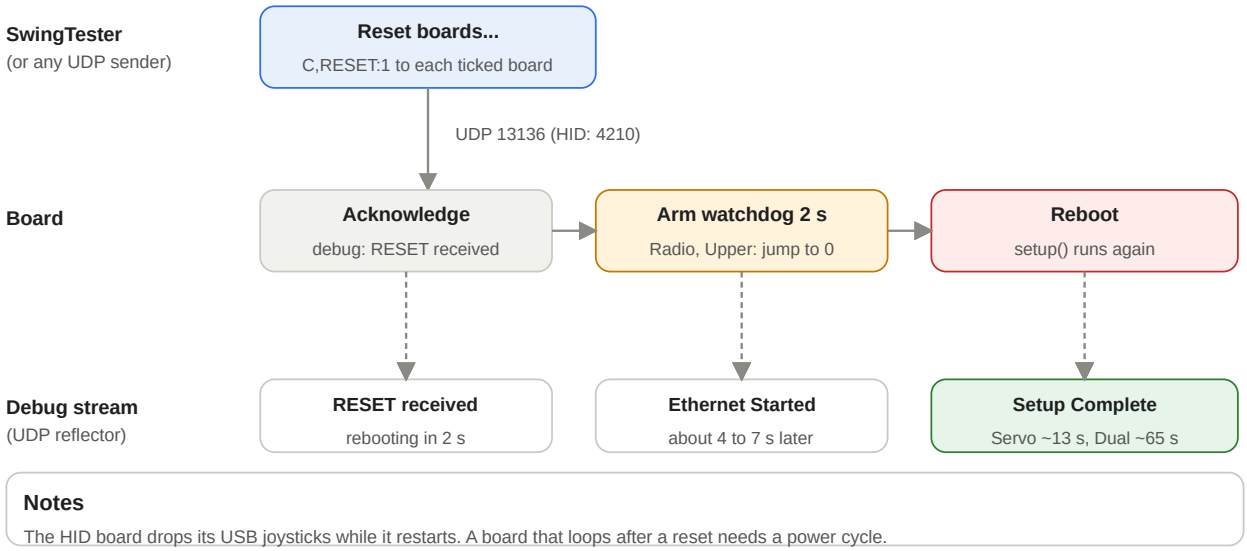


Figure 8 — C,RESET:1 on each board

The control packet `C,RESET:1` reboots a board about two seconds later.

- **Servo, Stepper, Dual Stepper and USB HID boards** arm the AVR watchdog for two seconds and spin until it fires. A short `.init3` function clears the watchdog first thing after every reset. The HID board listens on port 4210 (it also accepts the plain text `RESET`); its USB connection drops while it restarts, so Windows briefly loses the four joysticks and the keyboard.
- **Radio and Upper controllers** use a software reset (a jump to the reset vector after switching off interrupts and peripherals). A watchdog reset hung the Radio's bootloader, which restarts the watchdog at its shortest timeout and never reaches the sketch until it is reflashed.

If a board ever reboot-loops after a reset packet, power-cycle it and reflash. The SwingTester's *Reset boards...* button sends the command to the boards you tick.

> **Network note:** data that arrives in bursts with gaps of several seconds meant the PC's link to the board switch was dropping in and out. The cause on this cockpit was the 5 V rail feeding the network switch dipping below 4.75 V; the switch then dropped its ports. Check that supply first, then the cable and the switch port.

2.7 Debug and monitoring

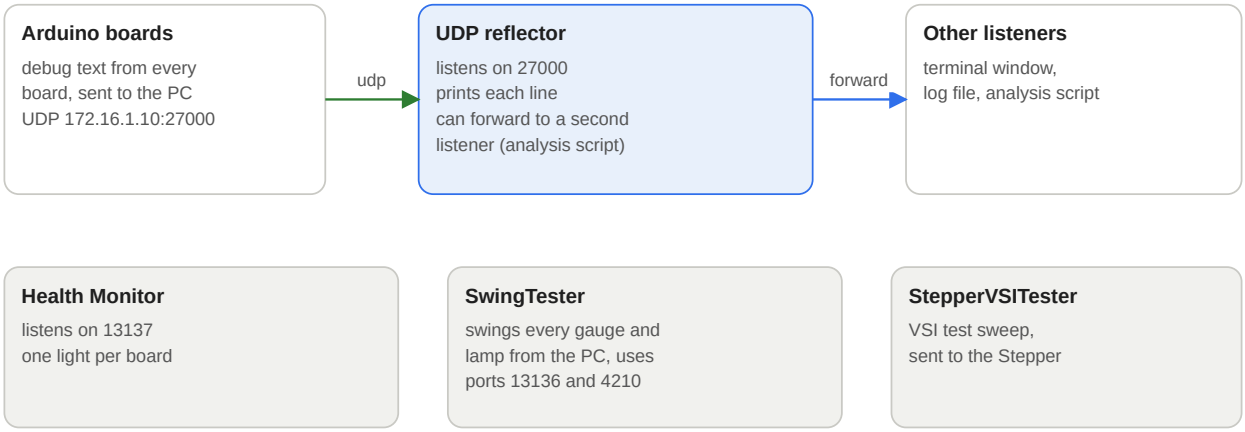


Figure 9 — Debug and test tools

Boards send debug text to `172.16.1.10:27000`. Only one program can own a UDP port, so if the reflector is running do not start a second listener on 27000; use the reflector's forwarding instead. The Health Monitor owns 13137 and each bridge owns 27001.

3 Radio controller (172.16.1.101)

`JET_RANGER_RADIO_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module. It shows the COM and NAV frequencies on two character OLEDs, reads eight rotary encoders (COM1, COM2, NAV1 and NAV2, each with a major and a minor knob) and a 4 by 4 button matrix, and keeps the radio panel in step with the simulator.

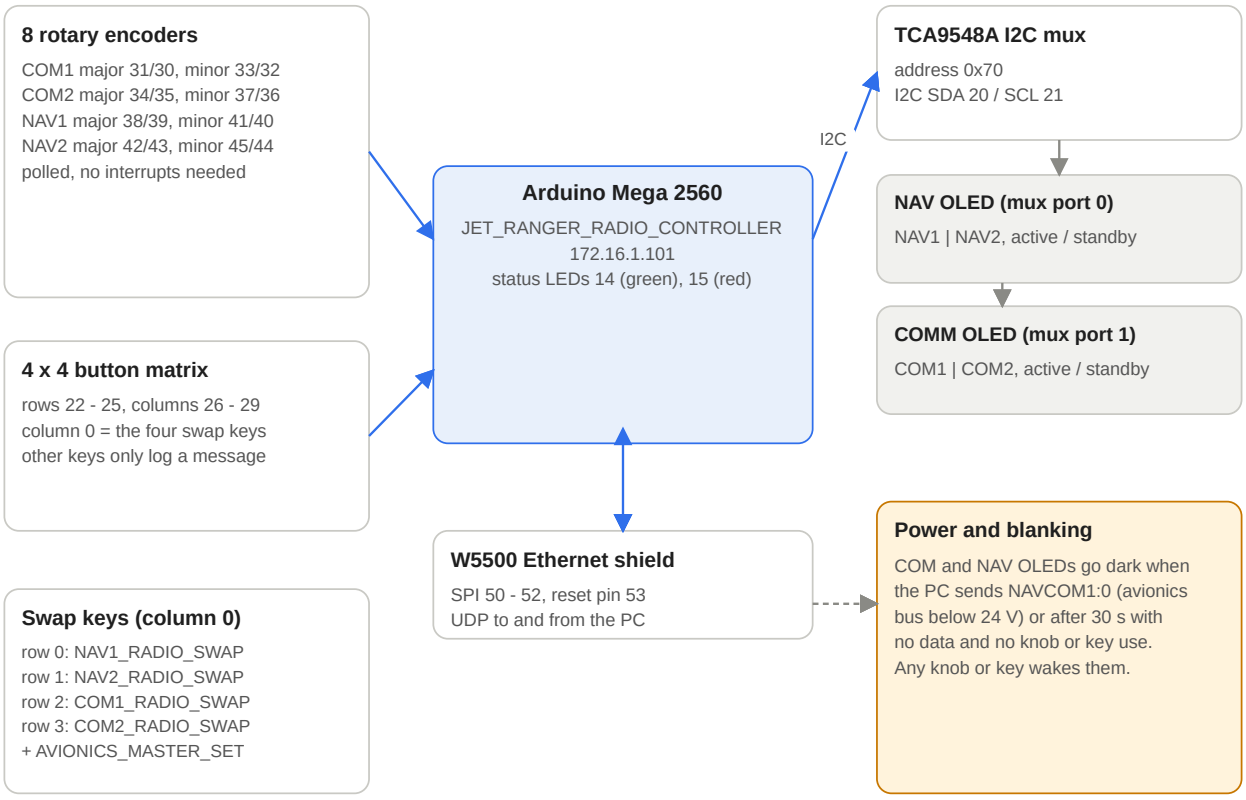


Figure 10 — Radio controller blocks

3.1 Program flow

1. **Setup:** reset the W5500 through pin 53, bring up Ethernet and open the UDP sockets, flash the red LED for 3 seconds while the link settles, send an "Ethernet Started" debug line, start the two OLEDs through the TCA9548A I2C multiplexer, start the eight encoders (with acceleration) and the matrix scanner.
2. **Main loop:**
    1. toggle the green and red status LEDs every 200 ms;
    2. every 10 seconds send the `COMM_NAV` heartbeat;
    3. poll the eight encoders; a change recalculates the standby frequency, sends it to the PC and redraws the OLEDs;
    4. scan the matrix and act on key presses (below);
    5. read any packet on port 13136 and update the active and standby frequencies, the main bus voltage and the power state.

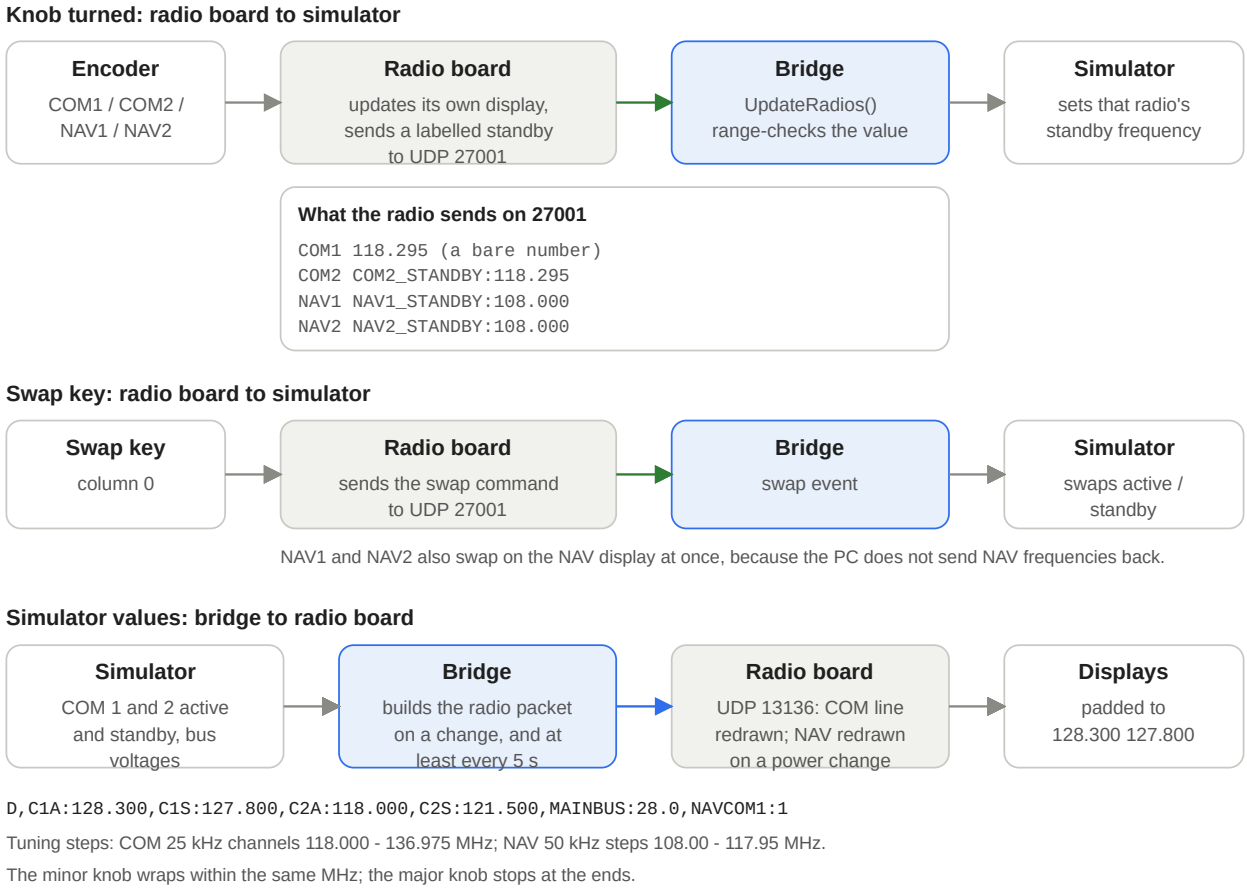


Figure 11 — Where each input and output goes

3.2 Pin usage

Pin	Function
14	Green status LED
15	Red status LED (flashes while the Ethernet link comes up)
53	W5500 manual reset
50 to 52 (and 10)	SPI to the W5500
SDA, SCL	TCA9548A mux (0x70): port 0 NAV OLED, port 1 COM OLED, both at 0x3C
31, 30	COM1 major encoder (A, B)
33, 32	COM1 minor encoder
34, 35	COM2 major encoder
37, 36	COM2 minor encoder
38, 39	NAV1 major encoder
41, 40	NAV1 minor encoder
42, 43	NAV2 major encoder
45, 44	NAV2 minor encoder
22 to 25	Matrix rows
26 to 29	Matrix columns

3.3 Data from the PC (UDP 13136)

Token	Meaning	Example
`C1A`, `C1S`	COM1 active and standby frequency	`C1A:119.500`
`C2A`, `C2S`	COM2 active and standby frequency	`C2S:121.900`

Token	Meaning	Example
MAINBUS	Main bus voltage	MAINBUS:27.8
NAVCOM1	Radio power: 1 = on, 0 = COM and NAV displays blank	NAVCOM1:1
C,B:n	Display brightness (control packet)	C,B:5

The bridges send COM frequencies with two decimals; the radio pads every frequency to xxx.xxx (7 characters) so that the 16-column line (7 + 2 + 7) never slides COM2 or NAV2 behind the faceplate. The PC does not send NAV frequencies back.

3.4 Commands to the PC (UDP 27001)

Turning a knob changes the standby frequency on the OLED and sends it to the PC. COM1 is sent as the bare number (for example 118.295); the others carry the radio's name: COM2_STANDBY:118.295, NAV1_STANDBY:108.000, NAV2_STANDBY:108.000. Nothing is sent during the first 1.5 seconds after power-up, so the knobs' start positions do not change the simulator.

The four frequency-swap switches are on the Jet Ranger COM and NAV PCB in a diode matrix that shares column 0 (D26); the rows are D22 to D25.

Switch	Matrix (row, col)	Action
NAV1 FREQ (SW6)	0, 0	Swaps NAV1 active and standby on the NAV OLED at once (the PC sends no NAV frequencies back) and sends NAV1_RADIO_SWAP
NAV2 FREQ (SW7)	1, 0	The same for NAV2, sends NAV2_RADIO_SWAP
COMM1 (SW4)	2, 0	Sends COM1_RADIO_SWAP; the display changes when the simulator returns the swapped frequencies
COMM2 (SW5)	3, 0	Sends COM2_RADIO_SWAP, then AVIONICS_MASTER_SET

Other matrix keys are only logged to the debug listener.

3.5 Tuning steps

The knobs follow standard channel spacing. COM runs 118.000 to 136.975 MHz in 25 kHz channels (major knob 118 to 136; minor knob 000, 025, 050, 075). NAV runs 108.00 to 117.95 MHz in 50 kHz steps (major 108 to 117). Each minor knob wraps round at its ends. The constants COM_MINOR_STEP_KHZ and NAV_MINOR_STEP_KHZ set the steps, and the wrap-around limits follow from them. 8.33 kHz channels are not offered. FSUIPC holds COM frequencies to 10 kHz, so in the simulator a channel such as 118.025 may appear as 118.02 or 118.03.

3.6 Power and blanking

- **Avionics power:** when the avionics bus is below 24 V the bridge sends NAVCOM1:0 and the COM and NAV displays blank together; they light together again when the bus rises.
- **No data for 30 seconds:** if no packet arrives and no knob or key is used, both OLEDs blank once. A packet, a knob movement or a key press restores them, and they blank again after another 30 seconds of silence.
- **Quiet period:** for 1.5 seconds (matrixQuietMs) after the matrix scanner starts it records state but raises no events, so a floating input cannot send false button presses at boot.

3.7 Diagnostics

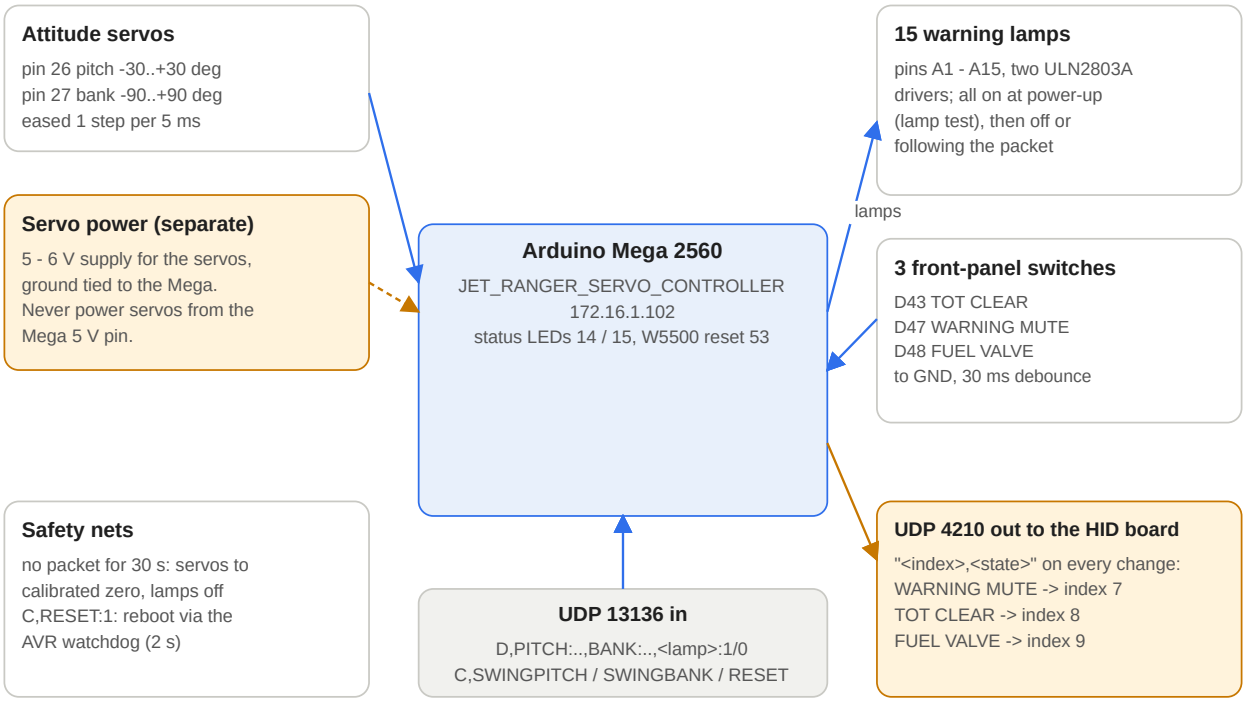
- The control packet C,MATRIX:1 prints the raw matrix (H or L for each column, with all rows high and then each row driven low) to the debug stream, bypassing debounce and the quiet period. A column that reads L with nothing pressed is a hardware fault on that Mega pin or board. A faulty or badly seated Mega did exactly this on the bench: all four swap switches share column 0, so none could register, and the board also dropped off USB and the network after flashing.
- Remote reset is a software reset (see the network chapter). Bench-tested: "Ethernet Started" about 4 seconds after the command and "Setup Complete" about 5 seconds.

3.8 Known issues

- Only COM1 and COM2 frequencies arrive from the PC; there are no NAV frequency tokens yet.
- The Radio controller emulator sends a bare frequency for every radio (see the emulator chapter).
- The mux address pins (D2 to D4) are not driven by the sketch; the mux module's own pull-downs set address 0x70.

4 Servo controller (172.16.1.102)

`JET_RANGER_SERVO_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the Jet Ranger Front Panel Servo PCB. It drives the attitude indicator's two servos (pitch and bank), switches the 15 caution and warning lamps and reads three front-panel switches. The other gauges that were once servo-driven (airspeed, vertical speed, torque, temperatures, pressures, fuel and RPM) now run on the two stepper boards.



Joystick 1 buttons 8, 9 and 10 in Windows (the firmware counts from 0). FSUIPC needs them bound to DS206 events.

Calibration: PITCH_DEG_TABLE -30 deg 179, 0 deg 113, +30 deg 70; BANK_DEG_TABLE -90 deg 5, 0 deg 93, +90 deg 179 (servo positions).

Figure 12 — Servo controller blocks

4.1 Program flow

1. **Setup:** set the status LED pins; light all 15 lamps as a lamp test; reset the W5500, start Ethernet and open the sockets (red LED flashes for 3 seconds); sweep pitch and then bank from minimum to maximum and back (about 2 seconds each) and park both at their calibrated zero; switch all lamps off; send the starting state of the three switches to the HID board; log "Setup Complete" and arm the 30 second watchdog.
2. **Main loop:**
    1. toggle the status LEDs every 200 ms and send the `SERV0` heartbeat every 10 seconds;
    2. every 5 ms check UDP 13136 and store each value as a new **target** (servo targets, lamp states);
    3. every 5 ms nudge each servo one step towards its target, so the attitude indicator moves smoothly; a servo that has not moved for 1 second is detached to stop buzz and re-attached on the next movement;
    4. service the three debounced switch inputs.

3. **Watchdog:** after 30 seconds with no packet both servos are eased back to their calibrated zero and all lamps go out. The remembered lamp states are reset as well, so a stale `1` cannot keep a lamp dark when the simulator next sends `1`.

4.2 Servos and calibration

Instrument	Mega pin	Token	Range
Attitude pitch	26	`PITCH`	-30 to +30 degrees
Attitude bank (roll)	27	`BANK`	-90 to +90 degrees

Both tokens take attitude in degrees with one decimal place allowed (for example `D,PITCH:4.5,BANK:-12`). The board converts degrees to a servo position (0 to 180) with two tables, interpolating in a straight line between points and holding the end value beyond them.

Instrument	Attitude	Servo position
Pitch (`PITCH_DEG_TABLE`)	-30 degrees	179
Pitch	0 (level)	113
Pitch	+30 degrees	70
Bank (`BANK_DEG_TABLE`)	-90 degrees	5
Bank	0 (level)	93
Bank	+90 degrees	179

`PITCHRAW` and `BANKRAW` take a raw servo position and skip the tables. The set-up sweep uses `aServMinPosition[]` and `aServMaxPosition[]` (bank 5 to 179, pitch 166 to 70) and parks at `aServZeroPosition[]` (bank 93, pitch 113). The PCB still has 16 servo headers from the earlier design; only pins 26 and 27 are used.

Calibrating a servo:

1. Send raw positions (`D,PITCHRAW:113`) until the instrument shows a known attitude.
2. Note the position for at least both ends and level.
3. Enter them in the table and set `aServZeroPosition[]` to the level position.
4. Upload, then check with real values (`D,PITCH:0`, `D,BANK:30`).

Do not drive a servo hard against its mechanical stop: a stalled servo draws high current and gets hot. Never power servos from the Mega's 5 V pin; use a separate 5 to 6 V supply with a common ground.

4.3 Warning lamps

The 15 warning lamps on the Servo board

Green: the bridge drives it from the simulator. Grey: needs aircraft-specific variables, so it stays off.

ENGINE OUT pin A1 token EOUT engine speed < 60 %	**ROTOR RPM LOW** pin A2 token RLOW rotor <= 95 %	**XMSN OIL PRESS** pin A3 token TOPW pressure < 40 psi
XMSN OIL TEMP pin A4 token TOTW always off	**BATTERY HOT** pin A5 token BHOT always off	**BATTERY TEMP** pin A6 token BTMP always off
ENGINE CHIP pin A7 token EC always off	**TAIL ROTOR CHIP** pin A8 token TRC always off	**XMSN CHIP** pin A9 token TC always off
BAGGAGE DOOR pin A10 token BD always off	**AFT FUEL FILTER** pin A11 token FFLTR always off	**FUEL PUMP** pin A12 token FPMP always off
GENERATOR FAIL pin A13 token GENF engine speed < 10 %	**LOW FUEL** pin A14 token LOWF fuel <= 17 gal	**SC FAIL** pin A15 token SCF always off

All lamps go out when the main bus is below 20 V. Sent as <token>:1 / <token>:0 in every Servo packet.
The lamp test at power-up lights all 15; a no-data timeout turns them all off.

Figure 13 — The 15 lamps and their Mega pins

Token	Lamp	Pin
EOUT	Engine out	A1
RLOW	Rotor RPM low	A2
TOPW	Transmission oil pressure	A3
TOTW	Transmission oil temperature	A4
BHOT	Battery hot	A5
BTMP	Battery temperature	A6
EC	Engine chip	A7
TRC	Tail rotor chip	A8
TC	Transmission chip	A9
BD	Baggage door	A10
FFLTR	Aft fuel filter	A11
FPMP	Fuel pump	A12
GENF	Generator fail	A13
LOWF	Low fuel	A14
SCF	SC fail (standby compass)	A15

A lamp pin is HIGH when lit (through a driver for lamps above 20 mA). The power-up lamp test lights every lamp, so it cannot show a mis-wired pin: send one lamp at a time (for example `D,RLOW:1`) to check each one.

4.4 Debounced switch inputs

Three front-panel switches are wired between a Mega pin and GND on the Front Panel Servo PCB and read with the internal pull-up (closed = pin LOW). A new reading must stay unchanged for 30 ms (`InputDebounceMs`) before it counts.

Pin	Switch	PCB net	HID index	Joystick 1 button in Windows
D43	TOT CLEAR (SW1)	/SW_TOT_CLEAR	8	9

Pin	Switch	PCB net	HID index	Joystick 1 button in Windows
D47	WARNING MUTE (SW3)	`/SW_WARNING_MUTE`	7	8
D48	FUEL VALVE (SW2)	`/SW_FUEL_VALVE`	9	10

Every accepted change is logged to the debug stream (for example `Jet Ranger Servo: WARNING MUTE CLOSED -> HID button 7`) and sent to the USB HID board at `172.16.1.11:4210` as `index,state` with 1 = closed. The HID board counts buttons from 0 and Windows from 1. The starting state of each switch is sent at the end of `setup()`; there is no periodic resend, so a HID board that boots later stays out of step until the switch next changes. Nothing is sent to the PC itself.

4.5 On-demand commands

- `C,SWINGPITCH:1` and `C,SWINGBANK:1` run one sweep (the value is ignored); the board pauses for about 2 seconds.
- `C,RESET:1` reboots the board through the AVR watchdog (bench-tested: "Ethernet Started" 6.9 seconds later, "Setup Complete" about 12.8 seconds after the command).

4.6 Pin usage

Pin	Function
14	Green status LED
15	Red status LED
26, 27	Pitch and bank servos
43, 47, 48	TOT clear, warning mute and fuel valve switches
53	W5500 reset
A1 to A15	Warning lamps
50 to 52	SPI to the Ethernet module

4.7 Known issues

- Brightness packets (`C,B:n`) are accepted but do nothing: no pin is assigned. The PCB has `BACKLIGHT_PWM` on pin 5 ready for it.
- `powerAvailable` is set from `NAVCOM1` but nothing uses it, so the attitude indicator and lamps stay live when the simulator's power is off (the bridge itself sends all lamps off below 20 V).
- Several lamps log "Transmission Temperature changed" to the debug stream whichever lamp changed.
- The `#define` comments for pins 26 and 27 still describe the old servo layout.
- The PCB labels 10 of the 15 lamp outputs on different Mega pins from the sketch (A1/A2, A5/A6, A7 to A10, A11/A12). Test each lamp on its own and correct whichever side is wrong.

5 Upper controller (172.16.1.103)

`JET_RANGER_UPPER_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module. It scans the overhead switch matrix (up to 256 positions, 176 used) and a small numeric keypad, and sends every change over UDP to the USB HID board, which presents them to Windows as joystick buttons and keystrokes. The sketch also contains DCS-BIOS code for DCS World over the USB serial link, but `CreateDcsBiosMessage()` is still a stub, so only the joystick path works.

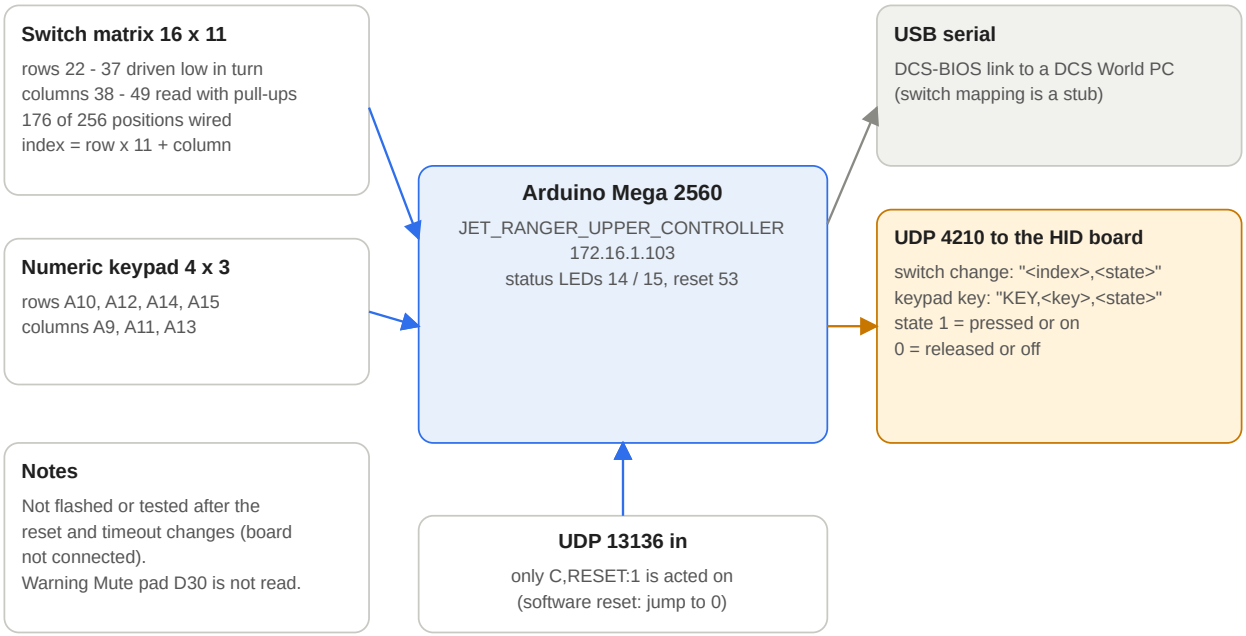


Figure 14 — Upper controller blocks

5.1 Program flow

1. **Setup:** set the status LEDs, reset the W5500, start Ethernet and open the sockets (red LED flashes for 3 seconds), configure pins 22 to 37 as matrix row outputs and 38 to 49 as column inputs with pull-ups, clear the button and debounce arrays and start the keypad.
2. **Main loop:**
    1. toggle the status LEDs every 200 ms and send the `UPPER_INPUT` heartbeat every 10 seconds;
    2. `ScanInputs()` drives each of the 16 rows low in turn with direct `PORTA` and `PORTC` writes and reads the columns with direct `PIND`, `PING` and `PINL` reads, building a 176-button state array;
    3. `FindInputChanges()` debounces and, for each change, calls `SendButtonToHid(index, state)`;
    4. `scanMatrix()` scans the 4 by 3 keypad and sends `SendKeyToHid(key, 1)` on press and `(key, 0)` on release;
    5. every 20 ms `ServiceControlPackets()` polls port 13136 and acts **only** on `C,RESET:1`; every other packet is ignored. The poll is slow on purpose so the extra W5500 access does not slow the matrix scan.

5.2 Pin usage

Pin	Function
14	Green status LED
15	Red status LED

Pin	Function
53	W5500 reset
22 to 37	16 matrix row drivers
38 to 49	Matrix column inputs; pins 50 to 53 are the Ethernet SPI bus, so the last four columns are fixed at 1 (unused)
A10, A12, A14, A15	Keypad rows
A9, A11, A13	Keypad columns
USB serial	DCS-BIOS link to a DCS World PC (separate from the Ethernet path)

5.3 What it sends

To	Port	Content
172.16.1.11 (USB HID board)	4210	`index,state` for matrix switches and `KEY,key,state` for keypad keys
172.16.1.10	27000	Debug text
172.16.1.10	13137	`UPPER_INPUT` heartbeat every 10 seconds

The Upper controller has its own address and MAC (`172.16.1.103`, `...83:03`); it never listens for simulator data, so no bridge sends to it.

5.4 Remote reset

`C,RESET:1` on port 13136 reboots the board with a software reset (see the network chapter). The code is the same as the Radio controller's and the sketch compiles, but it has **not been flashed or bench-tested** because the board was not connected.

5.5 Known issues

- The Upper controller PCB has a Warning Mute connector (J17) on `/ROW8`, Arduino D30, but the sketch's keypad uses analog pins A9 to A15, so J17 may not be read.
- `CreateDcsBiosMessage()` has no bodies, so none of these switches do anything in DCS.

6 USB HID board (172.16.1.11)

`UDP_TO_HID` runs on an Arduino with native USB (Leonardo class, board option `arduino:avr:leonardo`) and a W5500 Ethernet module. It is a network-to-USB bridge: it receives button and key commands over UDP from the Upper controller and the Servo controller and re-emits them to Windows as **four virtual 32-button joysticks (128 buttons) and a keyboard**. The joysticks have buttons only: no axes and no hat switches.

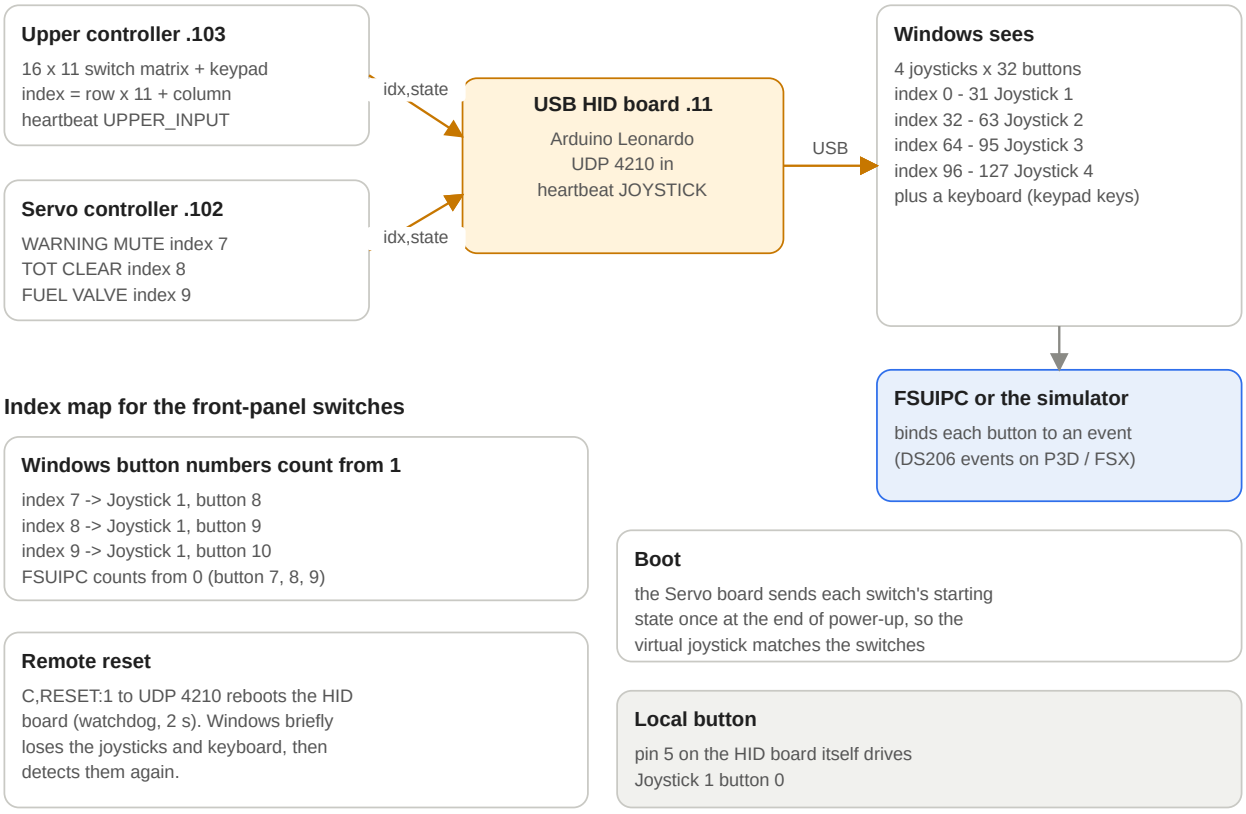


Figure 15 — From switch to joystick button

6.1 Program flow

1. **Setup:** start the four joysticks and the keyboard, bring up Ethernet on the static address, open the command, log and heartbeat sockets and log a boot message.
2. **Main loop:**
   1. send the `JOYSTICK` heartbeat every 10 seconds;
   2. read the one local push button on pin 5 and mirror it on Joystick 1, button 0;
   3. read any UDP command on port 4210, log the sender and payload and call `processCommand()`.

`processCommand()` understands three formats:

Payload	Effect
`index,state`	Sets button `index mod 32` of joystick `index div 32` (0 to 31 Joystick 1, 32 to 63 Joystick 2, 64 to 95 Joystick 3, 96 to 127 Joystick 4) pressed (1) or released (0)
`KEY,key,state`	Presses or releases a keyboard key. A single character is sent as its ASCII value; longer names (`ENTER`, `F5`, `LCTRL`) map to key codes
`C,RESET:1` (or `RESET`)	Reboots the board through the AVR watchdog; Windows briefly loses the joysticks and keyboard

6.2 How a switch becomes a joystick button

1. The Upper controller numbers each matrix position as `index = row x 11 + column` (0 to 175).
2. When a switch changes it sends `index,state` to `172.16.1.11:4210` with state 1 = switch on.
3. The HID board holds that button down while the switch is on.
4. Keypad keys are sent as `KEY,<key>,<state>` and typed as keyboard keys 0 to 9, `*` and `#`.
5. The Servo controller sends the same format for its three front-panel switches (indexes 7, 8 and 9).

Index range	Windows device
0 to 31	Joystick 1
32 to 63	Joystick 2
64 to 95	Joystick 3
96 to 127	Joystick 4
128 to 175	Not sent to Windows (out of range)

6.3 Switch mapping in use

From `UpperSwitchMapping.ods`. Button numbers are as Windows and the simulators show them (1 to 32); the firmware counts from 0. A latching switch holds its button down while on, so bind both press and release. The suggested events are the closest match in `DODSIM206V2.EVT`; the bindings actually saved in FSUIPC or MSFS are not in the repository. All other matrix positions are wired but unassigned.

Index	Source	Switch	Joystick 1 button	Suggested DodoSim event
0 to 4	Upper 0, 0-4	Heater rotary HTR 0 to 4	1 to 5	
5	Upper 0, 5	Air vent	6	`HEAT_VENT_SWITCH_SET`
6	Upper 0, 6	Air heat	7	`HEAT_VENT_SWITCH_SET`
7	Servo D47	Warning mute	8	`DS206.WARNING_MUTE_BUTTON_SET`
8	Servo D43	TOT clear (TOT light test)	9	`DS206.TOT_TEST_BUTTON_SET`
9	Servo D48	Fuel valve	10	`DS206.FUEL_VALVE_SWITCH_SET`
11	Upper 1, 0	Forward fuel boost breaker	12	`FORE_FUEL_PUMP_CB_SET`
12	Upper 1, 1	Aft fuel boost breaker	13	`AFT_FUEL_PUMP_CB_SET`
13	Upper 1, 2	Generator field breaker	14	`GENERATOR_FIELD_CB_SET`
14	Upper 1, 3	Generator breaker	15	`GENERATOR_RESET_CB_SET`
15	Upper 1, 4	Directional gyro / attitude indicator	16	`DIR_GYRO_SWITCH_SET`
16	Upper 1, 5	Battery	17	`BATTERY_SWITCH_SET`
17	Upper 1, 6	Generator (on / start)	18	`GENERATOR_SWITCH_SET`
22	Upper 2, 0	Landing light	23	`LANDING_LIGHT_SWITCH_SET`
23	Upper 2, 1	Instrument lights	24	`PANEL_LIGHTS_SWITCH_SET`
24	Upper 2, 2	Caution light breaker	25	`CAUTION_LT_CB_SET`
25	Upper 2, 3	Anti-collision light	26	`COLLISION_LIGHTS_SET`
26	Upper 2, 4	Position lights	27	`POSITION_LIGHTS_SET`
27	Upper 2, 5	Pitot heat	28	`PITOT_HEAT_SWITCH_SET`
28	Upper 2, 6	Avionics	29	`AVIONICS_SWITCH_SET`
39	Upper 3, 6	Strobe light	Joystick 2, 8	none yet

6.4 Pin usage and network

Item	Value
Pin 5	Local push button, Joystick 1 button 0
Static IP, MAC	`172.16.1.11`, `DE:AD:BE:EF:FE:ED`
Listen port	4210 (button, key and reset commands)
Log to	172.16.1.10:27000 (boot, button and key events, malformed commands)
Heartbeat to	172.16.1.10:13137, `JOYSTICK` every 10 seconds

The Leonardo is flashed on COM8 with `--fqbn arduino:avr:leonardo`. Bench-tested remote reset: "Joystick controller booted" about 4 seconds after the command.

6.5 Known issues

- The board's own test button on pin 5 also drives Joystick 1 button 1, the same button as HTR 0.
- The Servo controller's three switches use indexes 7, 8 and 9, which Windows shows as buttons 8, 9 and 10. If Windows buttons 7, 8 and 9 were wanted, button 7 is already the Air heat switch (index 6), so the indexes would have to move.
- After a reset the USB connection drops; re-check any game that has the joysticks open.

7 Stepper controller (172.16.1.105)

`JET_RANGER_STEPPER_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the Jet Ranger 13 Stepper PCB. It drives X27-style stepper gauges directly from the Mega's pins (no driver chips) and takes **real units**, converting them to needle steps with calibration tables in the sketch. The oil, transmission, RPM, load and altimeter gauges are on the Dual Stepper board (next chapter).

The sketch started as an A-10C front instrument board and still carries some DCS-BIOS code and an unused SARI roll stepper claim on pins 30 and 32; the Jet Ranger path is the UDP protocol.

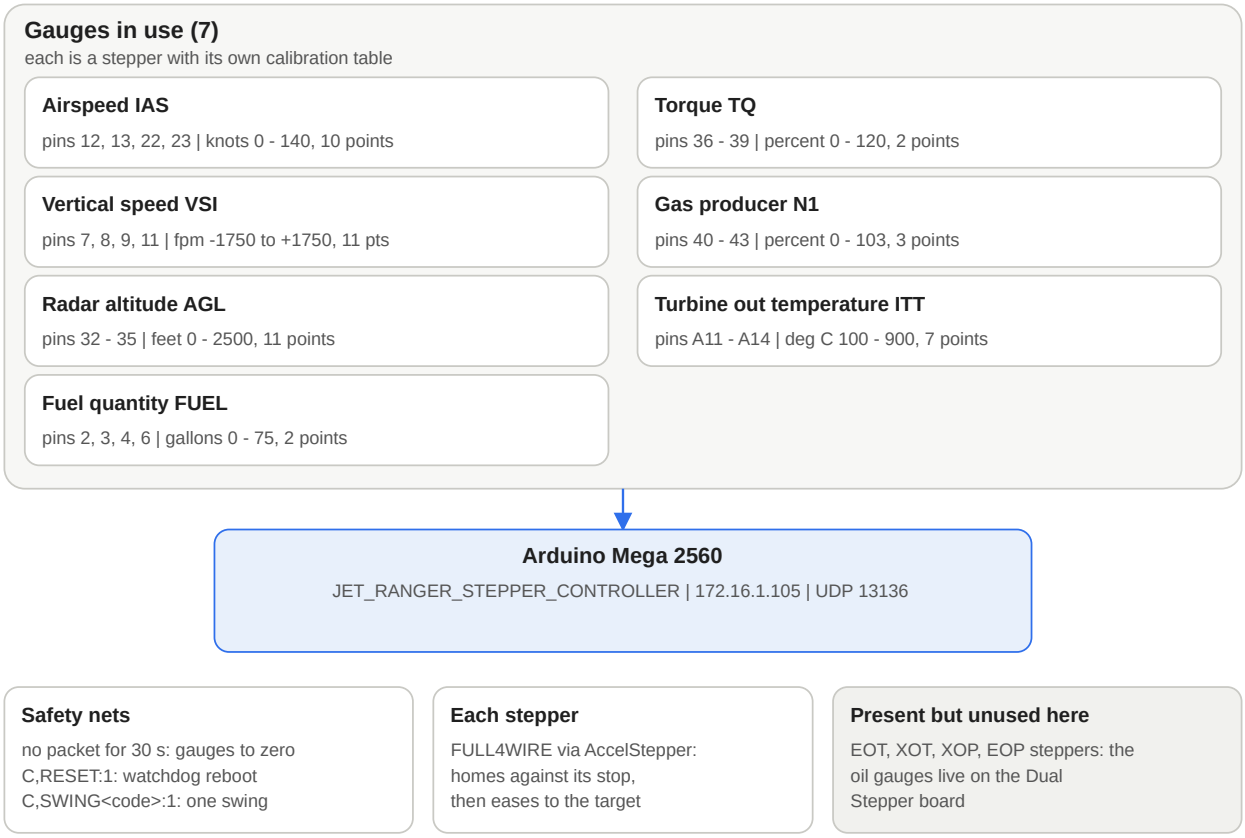


Figure 16 — Stepper controller blocks

7.1 Gauges and calibration

Gauge	Token	Mega pins	Unit and range	Calibration
Airspeed	`IAS`	12, 13, 22, 23	knots 0 to 140, one decimal allowed	`IAS_KT_TABLE`, 10 points (0 to 635 steps)
Vertical speed	`VSI`	7, 8, 9, 11	ft/min -1750 to +1750	`VSI_FPM_TABLE`, 11 points (-311 to +315 steps)
Radar altitude	`AGL`	32 to 35	feet 0 to 2500	`AGL_FT_TABLE`, 11 points (0 to 469 steps)
Torque	`TQ`	36 to 39	percent 0 to 120	`TQ_PCT_TABLE`, 2 points (0 to 600 steps)
Gas producer (N1)	`N1`	40 to 43	percent 0 to 103	`GP_PCT_TABLE`, 3 points (0 percent is step 15)
Turbine out temperature	`ITT`	A11 to A14	degrees C 100 to 900	`EGT_C_TABLE`, 7 points (100 C is step 0)
Fuel quantity	`FUEL`	2, 3, 4, 6	US gallons 0 to 75	`FA_GAL_TABLE`, 2 points (0 gal is step 53)

Notes:

- The first IAS row (0 kt = step 0) is an assumed zero. The VSI and IAS steppers are wired C, D, A, B.

- Every token has a raw twin (`IASRAW:300`) that moves the needle by raw steps and skips the table.
- Four more steppers are declared with placeholder linear scales and no start-up swing: engine oil temperature (`OILT`, pins 48, A0 to A2), transmission oil temperature (`XMSNT`, A3 to A6), transmission oil pressure (`XMSNP`, A7 to A10) and engine oil pressure (`OILP`, 44 to 47). The oil gauges are really driven by the Dual Stepper; wire each physical gauge to one board only.
- `RPME` and `RPMR` packets sent to this board are ignored.
- **Zero trims:** each gauge has a `<CODE>ZeroOffset` value (steps) to nudge its true zero without touching its table. Current values: `AGLZeroOffset` 10, `TQZeroOffset` 23, all others 0. The trim is applied in each gauge's real-value setter, so a `<CODE>:0` packet, the watchdog reset and the end of each swing all land on the same trimmed position. The `*RAW` codes do not apply it.

7.2 Program flow

1. **Setup:** flash the status LEDs, bring up Ethernet (10 ms retransmission timeout), ramp the backlight, then run each gauge's boot swing in this order, each behind its own `SwingXXX` gate: fuel quantity, torque, turbine out temperature, gas producer, VSI, airspeed, radar altitude. All seven gates are currently `true` (`SwingLoops` is 2). The 30 second watchdog is armed at the end of `setup()`.
2. **Main loop:** toggle the status LEDs, call `.run()` on every stepper, send the `STEPPER` heartbeat every 10 seconds, and every 5 ms check UDP 13136 and store new targets. Steppers move with smooth acceleration (maximum speed 9000, acceleration 1000).
3. **Watchdog:** after 30 seconds with no packet every gauge is driven to its calibrated zero through its own setter (trim included).

7.3 Packets

The bridges send this board its own packet:
`D,IAS:93.4,VSI:500,AGL:850,FUEL:42,TQ:65,ITT:610,N1:92`. `AGL` is zero below 20 V on the main bus. The fractional part of `IAS` is interpolated between table rows, so the needle moves smoothly rather than in whole-knot jumps.

7.4 On-demand swings

Send `C,<COMMAND>:1` (the value is ignored) to run one gauge's swing without rebooting. The swing blocks the loop for about 10 seconds and re-homes the stepper, so the needle then rests at its calibrated zero until the next packet.

Command	Gauge
`SWINGFUEL`	Fuel quantity
`SWINGTQ`	Torque
`SWINGITT`	Turbine out temperature
`SWINGN1`	Gas producer
`SWINGVSI`	Vertical speed
`SWINGIAS`	Airspeed
`SWINGAGL`	Radar altitude

The airspeed and radar altitude swings return to zero after the full-scale wind-up instead of 635 steps past it.

`C,RESET:1` reboots the board through the AVR watchdog (bench-tested: "Ethernet Started" 6.6 seconds later, a normal boot, no loop).

7.5 Pin usage

Pin	Function
14, 15	Green and red status LEDs
53	W5500 reset
5	Backlight PWM (`BACK_LIGHTS`; the PCB routes `BACKLIGHT_PWM` to D5 and VSI coil B to D8)
7, 8, 9, 11	VSI coils

Pin	Function
12, 13, 22, 23	Airspeed coils
32 to 35	Radar altitude coils
36 to 39	Torque coils
40 to 43	Gas producer coils
A11 to A14	Turbine out temperature coils
2, 3, 4, 6	Fuel quantity coils
48, A0 to A2	Engine oil temperature coils
A3 to A6	Transmission oil temperature coils
A7 to A10	Transmission oil pressure coils
44 to 47	Engine oil pressure coils
24 to 31	Unused (Turbine and Rotor speed moved to the Dual Stepper)

7.6 Network

Static IP `172.16.1.105`, MAC `A8:61:0A:67:83:69`, data port 13136, heartbeat `STEPPER` to 172.16.1.10:13137, debug to 172.16.1.10:27000. The StepperVSITester can send every token and raw token to this board without a simulator.

7.7 Known issues

- Pin 32 is claimed by both the radar altitude coil and the unused SARI roll stepper; pin A2 (56) is claimed by the engine oil temperature coil and the shared stepper enable pin. Neither SARI nor the enable pin is used.
- `VSIstepper.runToNewPosition(-X27_FULLWIRE_HOMING_STEPS)` expands to -635 + 5 = -630, not -640, because the macro is not parenthesised (about 1.5 percent).
- The placeholder oil and transmission gauges use a shared linear scale, not bench-measured tables.

8 Dual stepper controller (172.16.1.106)

`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER` runs on a third Mega 2560, on the Jet Ranger OLED Dual Stepper PCB. It drives stepper gauges for engine and transmission oil, engine and rotor RPM, fuel load, electrical load and the altimeter, plus **four small OLED displays**. Like the Stepper board it takes real units. It started as a copy of the Stepper sketch, which is why the two share their structure, their zero-trim approach and most of their code.

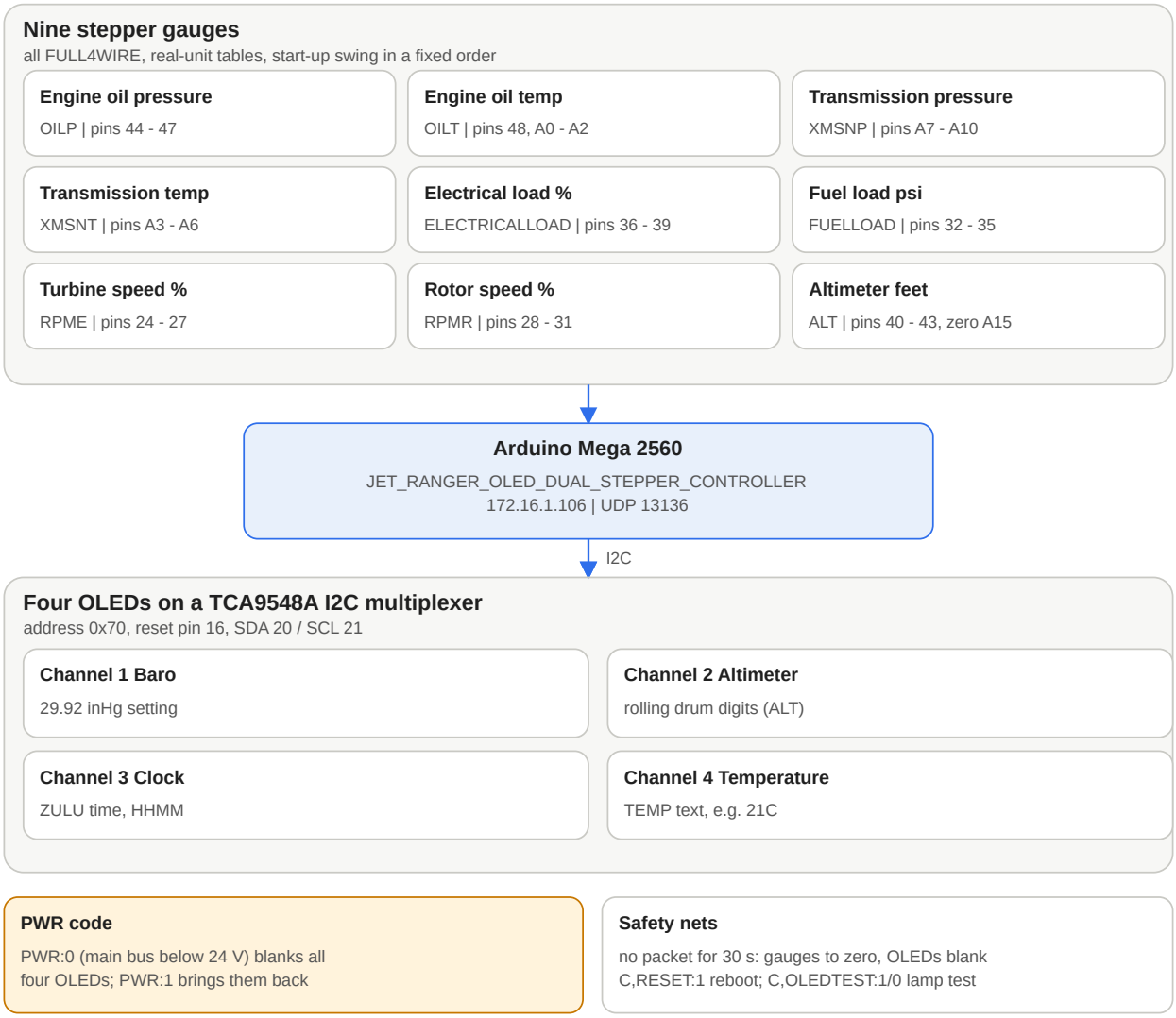


Figure 17 — Dual stepper controller blocks

8.1 Gauges and calibration

Gauge	Token	Mega pins	Unit	Calibration
Engine oil pressure	`OILP`	44 to 47	psi	`EOP_PSI_TABLE`: 0, 50, 100, 150 psi = 0, 74, 148, 220 steps
Engine oil temperature	`OILT`	48, A0 to A2	degrees C	`EOT_C_TABLE`: 0, 50, 100, 150 C = 0, 79, 156, 231 steps
Transmission oil pressure	`XMSNP`	A7 to A10	psi	`XOP_PSI_TABLE`: 0, 50, 70, 100, 150 psi = 0, 74, 104, 149, 220 steps
Transmission oil temperature	`XMSNT`	A3 to A6	degrees C	`XOT_C_TABLE`: 0, 50, 100, 150 C = 0, 75, 150, 225 steps
Electrical load	`ELECTRICALLOAD`	36 to 39	percent	`ELECTRICAL_LOAD_PCT_TABLE`: 0

Gauge	Token	Mega pins	Unit	Calibration
				percent = step 22, 100 percent = step 200
Fuel load	FUELLOAD	32 to 35	psi	FUEL_LOAD_PSI_TABLE: 0 psi = 0, 30 psi = 160 steps
Engine speed (N2)	RPME	24 to 27	percent, one decimal	TS_PCT_TABLE: 0, 55, 100, 110 percent = 0, 300, 535, 600 steps
Rotor speed	RPMR	28 to 31	percent, one decimal	RS_PCT_TABLE: the same values
Altimeter	ALT	40 to 43, zero sensor A15	feet	720 steps per 1000 ft

Turbine out temperature (ITT) and fuel quantity (FUEL) are also accepted, with the same tables as the Stepper board; wire each physical gauge to one board only. VSI, IAS, AGL, TQ and N1 are not handled here.

- Every token has a raw twin (FUELLOADRAW, RPMERAW, ALTRAW and so on).
- The 0 percent point of the RPM tables is assumed, and the 0 point of the electrical-load table is step 22, not zero.
- **Zero trims (steps):** TSoffset 10, RSoffset 0, EOTZeroOffset 24, EOPZeroOffset 0, XOTZeroOffset 16, XOPZeroOffset 5, FuelLoadZeroOffset 55, ElectricalLoadZeroOffset 24. The altimeter has none.
- The altimeter seeks its zero sensor on A15 by moving up to 10,080 steps (about 14 turns) before giving up, then sets position -25 and sweeps one 720 step revolution. The altimeter's own speed is 600 steps per second.

8.2 OLED displays

The diagram above shows the four displays on the multiplexer. Four 128 by 32 SSD1306 OLEDs share the Mega's I2C pins (20 SDA, 21 SCL) through a TCA9548A multiplexer at address 0x70. The firmware pulses the mux reset (pin 16) at start-up, calls Wire.begin() with a 25 ms timeout and probes the mux; if it does not answer the sketch logs "TCA mux NOT found - OLED init skipped" and starts the swings at once, then retries once after the swings.

Display	Mux channel	Token	Shows
Barometric setting	1	none yet	2992 at boot, not updated
Altimeter digits	2	ALT	Rolling-drum altitude, raw MSL (barometric correction ignored)
Clock	3	ZULU	Zulu time from HHMM, for example ZULU:1430 shows 14:30
Temperature	4	TEMP	The text as sent, for example TEMP:25C (up to 8 characters)

Updates are gated on change and throttled to every 300 ms. A boot-time I2C scan (OLED_I2C_SCAN) logs one line per channel to the debug stream, for example I2C scan ch1: 0x3c or I2C scan ch3: none; the latest scan found OLEDs on channels 1 to 4. The TCA9548A address pins (D17 to D19) are not driven by the sketch: the module's own pull-downs set address 0x70, so keep a module with pull-downs.

Power: PWR:0 (sent first in the bridge's packet while the main bus is below 24 V) blanks all four OLEDs and PWR:1 redraws them. The altimeter needle keeps following altitude. The lamp test ignores PWR.

Lamp test: C,OLEDTEST:1 fills all four OLEDs solid white and C,OLEDTEST:0 redraws their default content and clears the change gates so live data repaints them. It is ignored (with a debug line) if the mux was not found.

8.3 Program flow

1. **Setup:** initialise the OLEDs first so they are up while the needles swing, then run each gauge's swing in this order, each behind its own SwingXXX gate (all currently true, SwingLoops 1): engine oil pressure, engine oil temperature, transmission oil pressure, transmission oil temperature, electrical load, fuel load, engine speed, rotor speed, altimeter. Start-up takes about 65 seconds. The 30 second watchdog is armed at the end of setup().
2. **Main loop:** run the steppers, send the DUAL_STEPPER heartbeat every 10 seconds and check UDP 13136 every 5 ms.
3. **Watchdog:** after 30 seconds without a packet every needle returns to its calibrated zero and all OLEDs blank.

8.4 Packets

Bridge packet:
`D,PWR:1,ALT:1200,RPMR:100.0,RPME:100.0,ZULU:1430,OILT:60,OILP:55,XMSNT:50,XMSNP:35,FUELLOAD:17,ELECTRICALLOAD:50,TEMP:21C`. Rotor and engine speed carry one decimal so the fraction reaches the table interpolation.

8.5 On-demand commands

Command	Action
`SWINGOILP`, `SWINGOILT`, `SWINGXMSNP`, `SWINGXMSNT`	Oil and transmission gauges (about 5 to 6 seconds each)
`SWINGELECTRICALLOAD`, `SWINGFUELLOAD`, `SWINGRPME`, `SWINGRPMR`	Load and RPM gauges (about 6 to 7 seconds each)
`SWINGALT`	Altimeter (re-finds the zero sensor)
`OLEDTEST:1` / `OLEDTEST:0`	OLED lamp test on and off
`RESET:1`	Watchdog reboot (bench-tested: "Ethernet Started" 6.7 seconds later and a normal boot in about 56 seconds)

All are sent as `C,<COMMAND>:1`. The oil, transmission, load and RPM swings wind to the stop and return to the zero offset; the electrical-load swing returns to its 22 step zero.

8.6 Pin usage

Pin	Function
14, 15	Status LEDs
16	TCA9548A reset
20, 21	I2C SDA, SCL
53	W5500 reset
5	Backlight PWM (PCB `BACKLIGHT_PWM`)
24 to 27	Engine speed coils
28 to 31	Rotor speed coils
32 to 35	Fuel load coils
36 to 39	Electrical load coils
40 to 43, A15	Altimeter coils and zero sensor
44 to 47, 48, A0 to A2	Oil pressure and temperature coils
A3 to A10	Transmission temperature and pressure coils
2, 3, 4, 6, A11 to A14	Fuel quantity and turbine out temperature coils

8.7 Network

Static IP `172.16.1.106`, MAC `A8:61:0A:67:83:6A`, data port 13136, heartbeat `DUAL_STEPPER` (a prefix distinct from the Stepper's `STEPPER`, so the Health Monitor can tell them apart) to 172.16.1.10:13137, debug to 172.16.1.10:27000 including the I2C scan.

8.8 Known issues

- **RAM use is 75 to 77 percent** and the compiler warns "Low memory available". Avoid adding OLEDs or string-heavy code.
- The barometric OLED is initialised but never updated; nothing feeds it.
- The altimeter digits ignore barometric offset.
- Engine and rotor RPM tables assume 0 percent is 0 steps; `RSoffset` is not measured.
- The Clock OLED (channel 3) answered on the I2C scan but its display has not been confirmed by eye.
- **PCB version 1.4 (12 Oct 2026)** removes a short between Mega pins 2 and 32 (a copied net name joined fuel quantity coil A and fuel load coil A in copper). On a board older than 1.4 fit only one of those two gauges or cut the trace.

9 OLED controller (172.16.1.104)

`JET_RANGER_OLED_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module. Its header comments identify it as an A-10C front-console OLED sketch (`A10_FRONT_CONSOLE_OLED`): its main data source is DCS-BIOS over USB serial, not the Jet Ranger protocol. It is kept in the sketch folder because it can also be driven over UDP by the bridge, which sends it the altitude and barometric setting for two displays.

9.1 What it drives

Small OLEDs through a TCA9548A I2C multiplexer (address 0x70) and eight discrete lamps for an A-10C style console: a barometric-pressure readout, a large digital altimeter, a countermeasures status line, and landing-gear, VHF and angle-of-attack lamps. The altimeter and barometer are the only parts used by the Jet Ranger setup.

Mux port	Display
1	Barometer OLED
2	Altimeter OLED
3	Countermeasures (CMSC) character OLED at 0x3C

9.2 Program flow

1. **Setup:** flash the status LEDs, reset the W5500 and bring up Ethernet, set the eight lamp pins as outputs (off), start I2C, initialise the character OLED and show a test splash, scan all eight mux ports for I2C devices (results go to the debug stream), initialise the barometer and altimeter OLEDs, wait 3 seconds, turn the eight lamps on and start DCS-BIOS.
2. **Main loop:** toggle the status LEDs every 300 ms; run `DcsBios::loop()`; when the barometer setting changed recompute the altimeter correction and refresh both displays; every 5 ms check UDP 13136.

9.3 UDP interface

The sketch parses the same `D,CODE:value` packets as the Servo controller but handles only two codes; every other code is ignored.

Token	Effect
`ALT`	Redraws the rolling altitude digits (feet, unconverted)
`BARO`	Sets the four barometer digits from one value in inHg x 100 (`2992` is 29.92 inHg) and refreshes both displays

The FSUIPC bridge sends this board the whole shared front-panel packet and the board reads `ALT` and `BARO` out of it. The MSFS bridge sends it only those two codes.

9.4 Pin usage

Pin	Function
12	Red status LED
13	Green status LED
53	W5500 reset
SDA, SCL	TCA9548A mux
2	Right landing-gear lamp
3	Nose landing-gear lamp
4	Left landing-gear lamp
5, 6	VHF-AM and VHF-FM lamps
7, 8, 9	Angle-of-attack indexer lamps (above, on, below)
USB serial	DCS-BIOS link to a DCS World PC

Pin 2 is also used as an OLED reset pin in some declared but unused OLED objects; check the real wiring before reusing those code paths.

9.5 Network

Static IP `172.16.1.104`, MAC `A8:61:0A:67:83:68`, data port 13136, debug to 172.16.1.10:27000. The board sends no heartbeat, so the Health Monitor does not show it.

9.6 Notes

- `PTFont.h`, `dseg14_v3.h`, `er_oled.h` and `er_oled.cpp` in the sketch folder are not included by the sketch; they look like an earlier OLED driver.
- The Dual Stepper board now has its own altimeter and barometer OLEDs, so this board is optional for the Jet Ranger.

10 FSUIPC bridge (FSUIPCWinformsAutoCS)

`FSUIPCWinformsAutoCS` is the bridge for Prepar3D and FSX. It reads flight data through **FSUIPC** (raw memory offsets), sends each panel board its own packet, and listens on UDP 27001 for radio, battery and alternator commands from the Radio controller, acting on them through the FSUIPC API. It is a Windows Forms application (internally named `FSUIPCTest`); its window lists the values being sent and the 21 annunciator bits.

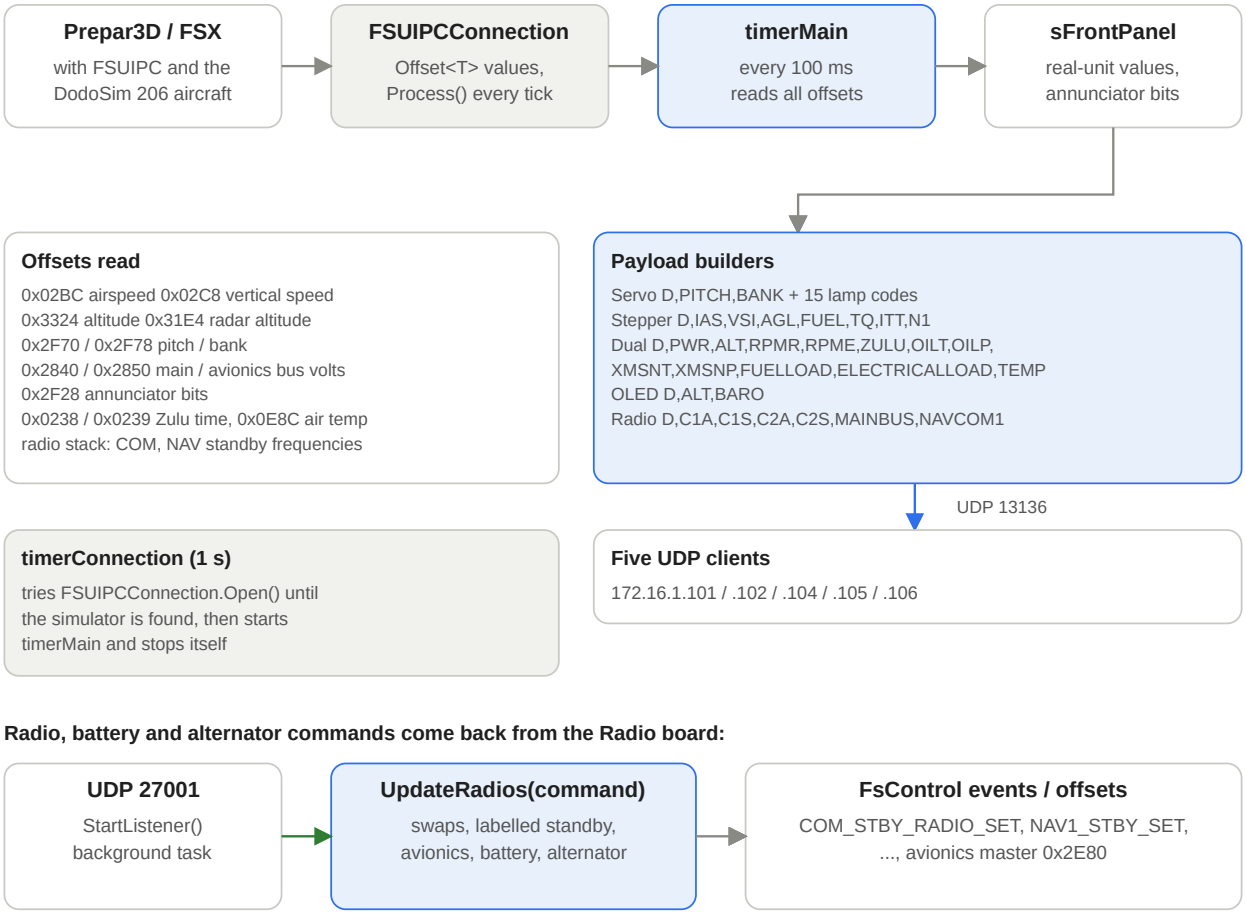


Figure 18 — FSUIPC bridge data flow

10.1 Program flow

1. **Constructor:** build the live annunciator list (one row per bit of the 0x2F28 field), connect the UDP clients and start `timerConnection`, which polls once a second for FSUIPC. Start the UDP listener on port 27001 in a background task. (The UDP client `.Connect` calls live in `Form1.Designer.cs`.)
2. **`timerConnection_Tick`:** keep trying `FSUIPCConnection.Open()`. Once it succeeds, stop and start `timerMain`.
3. **`timerMain_Tick`** (about 100 ms, so roughly 10 packets per second per board):
   1. `FSUIPCConnection.Process()` refreshes every declared offset and the radio-stack group;
   2. format the on-screen readout and update the annunciator list;
   3. build and send each board's packet (below);
   4. send the radio packet when a frequency or the bus voltage changed (at most every 200 ms) or every 5 seconds regardless.
4. **`UpdateRadios(command)`** handles port 27001 (below).

5. **chkAvionicsMaster_CheckedChanged** writes the avionics master offset when the operator toggles the checkbox, and closing the window stops both timers and the FSUIPC connection.

10.2 FSUIPC offsets

Offset	Meaning
0x02BC	Airspeed (knots x 128, so finer than 0.1 knot)
0x02C8	Vertical speed
0x0330	Altimeter pressure setting, hPa x 16 (sent on as BARO, converted to inHg x 100)
0x0238, 0x0239	Zulu hours and minutes (combined into the HHMM ZULU)
0x0E8C	Outside air temperature, degrees C x 256 (sent as TEMP, for example 21C; a standard FSUIPC offset, not verified against DodoSim)
0x08F0	Turbine-out percent and temperature
0x08BA, 0x08B8	Engine oil pressure and temperature
0x0900, 0x0904	Transmission oil pressure and temperature
0x0B74, 0x0B78	Fuel percent quantity and fuel capacity
0x08F8	Fuel pressure
0x08F4	Torque percent
0x0898	Gas producer
0x0908, 0x0896	Rotor RPM and engine N2
0x282C	Battery load amps
0x3324	Altitude
0x31E4	Radio altimeter
0x2F70, 0x2F78	Attitude pitch and bank
0x2F28	21-bit annunciator (warning lamp) bit field
0x2840, 0x2850	Main bus and avionics bus voltage
0x2E80	Avionics master (read and write)
RadioStack group	COM1 and COM2, NAV1 and NAV2 active and standby, transponder, ADF

10.3 Packets sent

The Servo, Stepper and Dual Stepper boards each get their **own** packet in real units; the boards convert with their own calibration tables. Whole numbers are rounded.

Board	Packet contents
Servo .102	PITCH (degrees, one decimal, negated because FSUIPC pitch is nose-down positive), BANK (degrees, one decimal), the 15 lamp codes EOUT, TOPW, BHOT, EC, BD, GENF, TOTW, RLOW, FFLTR, BTMP, TC, TRC, FPMP, LOWF, SCF as :1 or :0
Stepper .105	IAS (knots, one decimal), VSI (ft/min), AGL (feet, power-gated), FUEL (US gal), TQ (%), ITT (degrees C), N1 (%)
Dual Stepper .106	PWR (first: 1, or 0 while the main bus is below 24 V), ALT, RPMR, RPME (one decimal), ZULU, OILT, OILP, XMSNT, XMSNP, FUELLOAD (psi, from fuel pressure), ELECTRICALLOAD (%), TEMP
OLED .104	The shared front-panel packet; the board reads ALT and BARO out of it and ignores the rest
Radio .101	C1A, C1S, C2A, C2S, MAINBUS, NAVCOM1 (1 when the avionics bus, offset 0x2850, is 24 V or more)

10.4 Commands from the Radio controller (UDP 27001)

Command	Action
COM1_RADIO_SWAP, COM2_RADIO_SWAP	FSUIPC controls COM_STBY_RADIO_SWAP and COM2_RADIO_SWAP
NAV1_RADIO_SWAP, NAV2_RADIO_SWAP	FSUIPC controls NAV1_RADIO_SWAP and NAV2_RADIO_SWAP
AVIONICS_MASTER_SET	Writes the avionics master offset 0x2E80
MASTER_BATTERY_ON, MASTER_BATTERY_OFF	FSUIPC only has a toggle, so the bridge reads the main bus

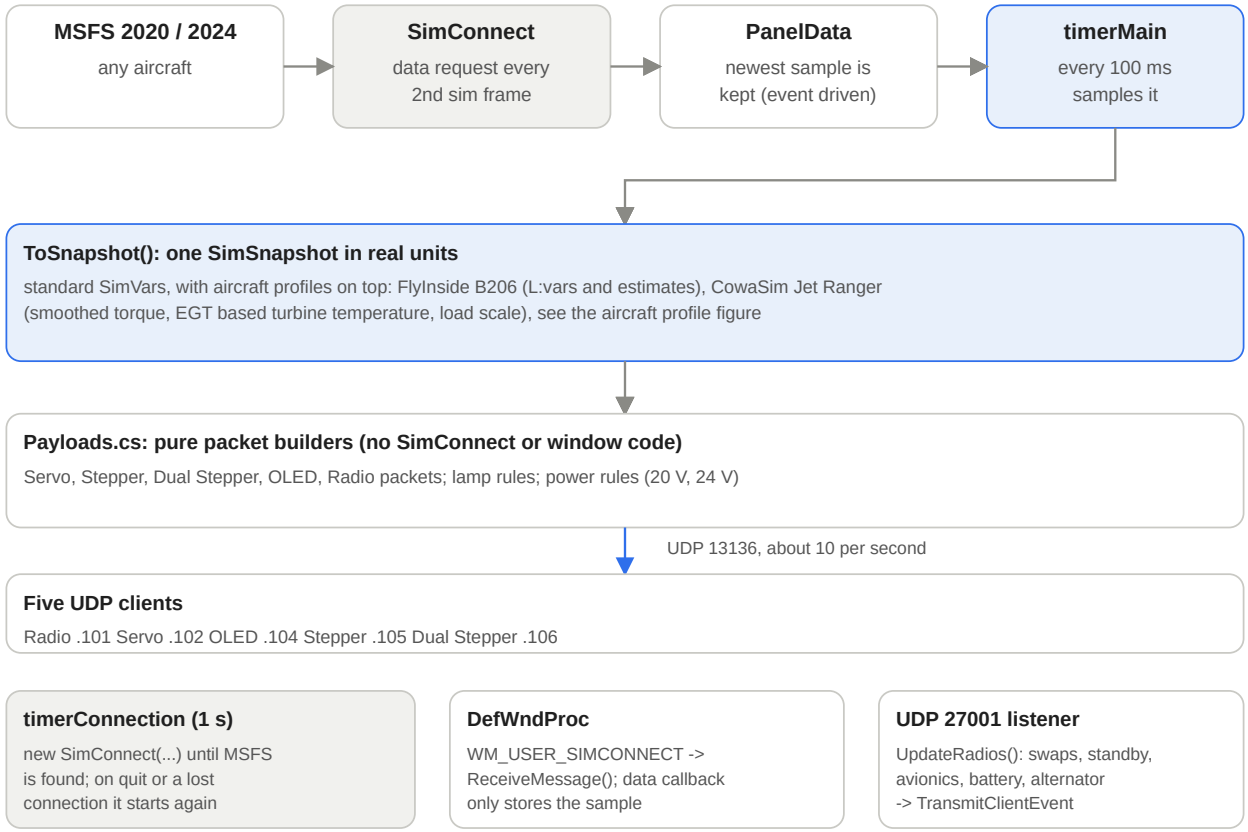
Command	Action
	voltage and toggles only when that would move the switch the requested way
`ALTERNATOR_ON`, `ALTERNATOR_OFF`	`TOGGLE_MASTER_ALTERNATOR` unconditionally (no alternator state is read)
`COM2_STANDBY:`, `NAV1_STANDBY:`, `NAV2_STANDBY:` plus a frequency	Tunes that radio's standby frequency
A bare number from 118.00 to 136.975	COM1 standby: BCD-encoded and sent with `COM_STBY_RADIO_SET`. Any other unrecognised string ends up in this parser too

10.5 Notes and quirks

- Several unused annunciator bits (`SPARE_*`, `SIMCONNECT_FAIL`, `LOW_INLET_PRESSURE`) all share the short code `SCF`, so the last one in the packet wins for that lamp.
- `TURB_ENG_ITT_1` is assigned twice per tick (percent, then temperature), so the temperature wins.
- Run only one bridge at a time; the MSFS bridge also listens on UDP 27001.
- Built clean and run against Prepar3D v6.1 sending about 10 packets per second per board.

11 MSFS bridge (MSFSWinformsAutoCS)

`MSFSWinformsAutoCS` is the bridge for Microsoft Flight Simulator 2020 and 2024. It has the same window, the same timers, the same UDP endpoints and the same real-unit packet per board as the FSUIPC bridge, but reads the data through **SimConnect**. For aircraft whose cockpit dials do not read the standard simulator variables it applies estimates fitted to dial readings (a FlyInside B206 profile and a CowaSim Jet Ranger profile).



No new data for 3 s (paused or loading): the bridge stops sending and the boards' 30 s watchdogs zero the gauges.

Run only one bridge at a time: FSUIPCWinformsAutoCS and MSFSWinformsAutoCS both listen on UDP 27001.

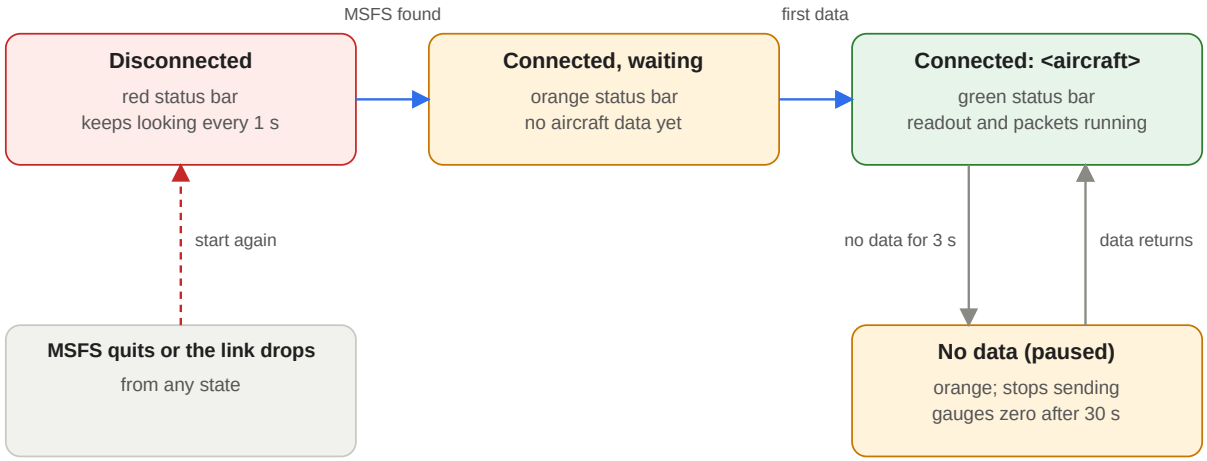
Figure 19 — MSFS bridge data flow

11.1 Files

File	Role
`Form1.cs`	SimConnect connection, 100 ms main timer, on-screen readout, lamp list, UDP listener and command handler
`Payloads.cs`	`SimSnapshot` (one sample of real-unit values) and `Payloads` (packet builders, lamp rules, power thresholds); no SimConnect or window code, so it can be tested on its own
`Form1.Designer.cs`, `Program.cs`	Window layout (identical to the FSUIPC bridge) and entry point
`lib\Microsoft.FlightSimulator.SimConnect.dll`, `lib\SimConnect.dll`	Managed wrapper and native library copied from the MSFS 2024 SDK so the project builds without the SDK path
`MSFSWinformsAutoCS.csproj`	`net10.0-windows10.0.19041.0`, WinForms

11.2 Program flow

1. **Constructor:** connect five UDP clients (port 13136), build the lamp list, start the 1 second `timerConnection` and the UDP listener on 27001.
2. **`timerConnection_Tick`:** try `new SimConnect("MSFS Winforms Bridge", Handle, WM_USER_SIMCONNECT, ...)`. While MSFS is not running this throws and the timer tries again. On success it registers the data definition, requests it every second sim frame, maps the client events, starts `timerMain` and stops itself.
3. **`DefWndProc`** passes `WM_USER_SIMCONNECT` to `ReceiveMessage()`; the data callback only stores the newest sample. If MSFS quits or the connection throws, the app closes SimConnect, stops `timerMain` and goes back to step 2.
4. **`timerMain_Tick`** (100 ms) converts the newest sample to a `SimSnapshot`, updates the readout, lamp list and Avionics Master box, then sends the packets: Radio when a frequency, the main bus voltage or the radio power flag changed (at most every 200 ms) or every 5 seconds; one packet each to the Servo, Stepper, Dual Stepper and OLED boards about every 100 ms.
5. **`UpdateRadios`** (UDP 27001) handles the Radio controller's commands as SimConnect client events.



A new aircraft is picked up while connected: the name in the status bar changes and the estimate models restart.

SimConnect errors are caught on every path, so a closed simulator never stops the bridge. FSUIPCWinformsAutoCS behaves the same way with FSUIPC.

Figure 20 — Connection states

If no new data has arrived for 3 seconds (paused or loading) the bridge stops sending, so the boards' 30 second watchdogs return the gauges to zero, and the status bar says so. The status bar also shows the aircraft title and follows it when a different aircraft is loaded.

11.3 Packets

Board	Packet
Servo .102	`PITCH` (degrees, one decimal, negated: the simulator reports nose-down positive), `BANK`, and the 15 lamp codes (all `0` while the main bus is below 20 V)
Stepper .105	`IAS` (knots, one decimal), `VSI`, `AGL` (feet, 0 below 20 V), `FUEL` (US gal), `TQ`, `ITT` (degrees C), `N1`
Dual Stepper .106	`PWR` (1, or 0 while the main bus is below 24 V), `ALT`, `RPMR`, `RPME`, `ZULU`, `OILT`, `OILP`, `XMSNT`, `XMSNP`, `FUELLOAD` (fuel pressure, psi), `ELECTRICALLOAD` (percent, below), `TEMP`
OLED .104	`ALT`, `BARO` (inHg x 100 from the Kohlsman setting in millibars) only
Radio .101	`C1A`, `C1S`, `C2A`, `C2S`, `MAINBUS`, `NAVCOM1` (1 when the avionics bus is 24 V or more; if the aircraft reports the avionics bus as exactly 0, as the FlyInside B206 does, the main bus is used instead)

`ELECTRICALLOAD` is the magnitude of `ELECTRICAL TOTAL LOAD AMPS` divided by 300 A (the sign goes negative while the battery supplies the bus, for example with the engine stopped); the CowaSim profile uses 139 A.

11.4 SimVars requested

`PLANE ALTITUDE`, `AIRSPEED INDICATED`, `VERTICAL SPEED`, `PLANE ALT ABOVE GROUND`, `ATTITUDE INDICATOR PITCH` and `BANK DEGREES`, `ROTOR RPM PCT:1`, `GENERAL ENG PCT MAX RPM:1`, `ENG TORQUE PERCENT:1`, `TURB ENG MAX TORQUE PERCENT:1`, `ENG EXHAUST GAS TEMPERATURE:1`, `GENERAL ENG THROTTLE LEVER POSITION:1`, `ELECTRICAL TOTAL LOAD AMPS`, `TURB ENG ITT:1`, `ENG OIL TEMPERATURE:1`, `FUEL TOTAL QUANTITY`, `TURB ENG CORRECTED N1:1`, `ENG OIL PRESSURE:1`, `ENG TRANSMISSION PRESSURE:1`, `ENG TRANSMISSION TEMPERATURE:1`, `GENERAL ENG FUEL PRESSURE:1`, `ELECTRICAL MAIN BUS VOLTAGE`, `ELECTRICAL AVIONICS BUS VOLTAGE`, `AVIONICS MASTER SWITCH`, `AMBIENT TEMPERATURE`, `KOHLSMAN SETTING MB`, `ZULU TIME`, and the COM 1, COM 2, NAV 1 and NAV 2 active and standby frequencies. `PanelVars` and the `PanelData` struct must list them in the same order; `CheckDefinitionMatchesStruct()` checks the count at connect.

11.5 Commands from the panel (UDP 27001)

`COM1_RADIO_SWAP`, `COM2_RADIO_SWAP`, `NAV1_RADIO_SWAP`, `NAV2_RADIO_SWAP`; `COM2_STANDBY:`, `NAV1_STANDBY:`, `NAV2_STANDBY:` plus a range-checked frequency (sent as `COM2_STBY_RADIO_SET_HZ`, `NAV1_STBY_SET_HZ`, `NAV2_STBY_SET_HZ`); a bare number is the COM1 standby frequency (an out-of-range or unreadable value becomes 121.50); `AVIONICS_MASTER_SET`; `MASTER_BATTERY_ON` and `_OFF` (`MASTER_BATTERY_SET` 1 or 0); `ALTERNATOR_ON` and `_OFF` (`ALTERNATOR_SET`). Unlike FSUIPC, SimConnect has direct set events, so there is no voltage guard or blind toggle.

11.6 Aircraft profiles

Value	Standard SimVars	FlyInside B206	CowaSim Jet Ranger
Rotor, engine speed	**ROTOR RPM PCT,** GENERAL ENG PCT MAX RPM	**L:Aircraft.Rotor.RPM.Percent,** L:...Turbine.N2.Smooth	**standard**
Gas producer (N1)	**TURB ENG CORRECTED N1**	**0.92 x turbine N2**	**standard**
Torque (TQ)	**ENG TORQUE PERCENT**	**collective lever curve,** up to 120 %	**0.42 x SimVar,** smoothed 2.5 s
Turbine out temp (ITT)	**TURB ENG ITT**	**6 x N2 + 1.05 x** (torque - 21)	**EGT - 200 C**
Fuel load (psi)	**ENG FUEL PRESSURE**	**10 x SimVar**	**standard**
Electrical load (%)	**|total amps| / 300 A**	**|total amps| / 300 A**	**|total amps| / 139 A**
Transmission pressure	**SimVar (usually 0)**	**35 psi x rotor fraction**	**48 psi x rotor fraction**
Transmission temp	**SimVar (usually 0)**	**slow warm-up from air temp**	**0.92 x engine oil temp**
Recognised by	**otherwise**	**turbine L:var, or title** contains FlyInside	**title contains 206B3**

Green = the standard SimVar is used unchanged. Estimates come from a few dial readings and may be a few percent off.

Figure 21 — Which values come from where

Other aircraft use the standard SimVars unchanged. Estimates were fitted to a few dial readings taken from screenshots and may be a few percent off at power settings that were not checked.

11.6.1 FlyInside B206

The FlyInside B206 does not feed the standard rotor and engine speed SimVars (they read 0 and a slowly decaying idle while the cockpit shows 100 percent). Found with Developer Mode, Tools, Behaviors, Variables (Global tab, filter `Aircraft.`): the cockpit uses `L:Aircraft.Rotor.RPM.Percent` and `L:Aircraft.Engine.1.Turbine.N2.Smooth`. The aircraft is recognised by the turbine L:var being above zero or by `FlyInside` in the aircraft title. The bridge uses the L:vars when above zero, otherwise the standard SimVars. They feed `RPMR`, `RPME` and the `RLOW` and `EOUT` lamps.

Value	Estimate
Gas producer (`N1`)	0.92 x turbine N2 (no variable found; the standard N1 reads about 52 while the cockpit shows 92)
Torque (`TQ`)	`ENG TORQUE PERCENT:1` reads 0. With the turbine at 95 or more: 21 + 0.72 x collective lever up to lever 73.6 (dial 74), then about 1.74 per lever percent to 120 at lever 100, clamped 0 to 120. Below that speed it is estimated from `TURB ENG MAX TORQUE PERCENT:1` through two readings
Turbine out temperature (`ITT`)	6 x turbine N2 + 1.05 per torque percent above the idle torque of 21 (the standard value reads about 540 while the dial shows 600)
Fuel load (`FUELLOAD`)	10 x `GENERAL ENG FUEL PRESSURE:1` (the cockpit dial is a 0 to 3 scale marked x10)

Value	Estimate
Transmission pressure (XMSNP)	35 psi x the rotor speed fraction
Transmission temperature (XMSNT)	Starts at outside air temperature and moves slowly (5 minute warm-up, 10 minute cool-down) toward ambient + 0.8 per torque percent while the rotor is above 50 percent; restarts from ambient when the bridge restarts



Both curves were fitted to dial readings from screenshots; check them against the cockpit at other power settings.

FlyInside above lever 73.6 climbs at about 1.74 points per percent; the engine over-torques if full collective is held for 2 s.

Figure 22 — Torque curves

Holding full collective for more than about 2 seconds over-torques and breaks the engine in this aircraft (the dial swings to about 115 percent). The lever mapping depends on the controller; only two steady readings back the 0.72 slope.

11.6.2 CowaSim Jet Ranger

The CowaSim Jet Ranger (aircraft title `206B3 ...`, for example `206B3 C-GTOK`) reports the standard SimVars and most dials match them. Gas producer, engine oil, fuel pressure and rotor and engine speed need no change. Three dials do not read the standard values, and it has no transmission oil values:

- **Torque:** the dial is a smoothed copy of `ENG TORQUE PERCENT:1`, about 0.42 x the SimVar (dial about 38 at the steady idle value of 83, about 120 at a peak of about 260) with a smoothing time constant of about 2.5 seconds. Error is about 18 points on a signal swinging between 0 and 120.
- **Turbine out temperature:** the dial follows `ENG EXHAUST GAS TEMPERATURE:1`, not ITT (which reads over 2,000): EGT minus 200 C, within about 30 C over nine readings.
- **Electrical load:** the LOAD dial sat at about 50 percent while the total load was 69.5 A, so 139 A is 100 percent (one reading).
- **Transmission oil:** pressure is 48 psi x the rotor speed fraction (the dial read about 45 to 50 psi); temperature is 0.92 x the engine oil temperature (one reading, which did not follow a slow warm-up).

11.7 Warning lamps

Five lamps are driven; the other ten are aircraft specific (DodoSim supplies them through an FSUIPC offset, which MSFS aircraft do not have) and stay off. Add rules to `Payloads.LampState()` once an aircraft's variables are known.

Lamp	Rule
EOUT engine out	Turbine speed below 60 percent
RLOW rotor low	Rotor at or below 95 percent
LOWF low fuel	17 US gal or less
TOPW transmission oil pressure	Below 40 psi, only when a pressure is available (the FlyInside and CowaSim

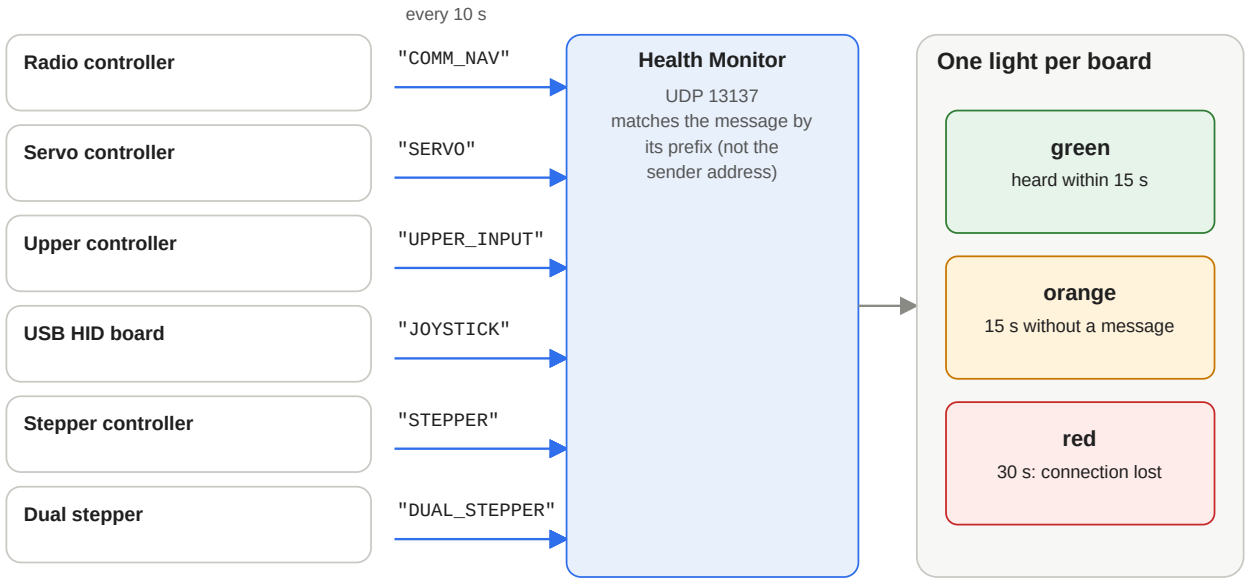
Lamp	Rule
	estimates, or an aircraft whose SimVar reads above 0)
GENF generator fail	Engine speed below 10 percent

11.8 Known gaps

- The FlyInside and CowaSim estimates come from a few readings and have not been checked at other power settings.
- Some MSFS Jet Ranger add-ons do not report every SimVar (transmission pressure and temperature read 0 on two of them; one reports a fixed rotor speed). A missing avionics bus voltage (exactly 0) falls back to the main bus for the Radio power rule.
- Pitch sign and the 24 V thresholds are carried over from the FSUIPC bridge and are not verified for every MSFS aircraft.
- There is no hysteresis on the 20 V and 24 V thresholds.
- The FlyInside electrical load dial stays at about 50 percent whatever the load.
- The FlyInside transmission pressure estimate is 35 psi at normal rotor speed, which is below the 40 psi lamp threshold, so the TOPW lamp stays on with that aircraft.

12 Health Monitor (JetRangerHealthMonitor)

A small Windows Forms dashboard (namespace `MegaHealthMonitor`) that watches for the 10 second heartbeat strings sent by the networked Arduino boards and shows one traffic light per board: Comm/Nav, Servo, Joystick, Upper Input, Stepper and Dual Stepper.



The monitor only listens; it never sends anything. It does not care which bridge is running.

Figure 23 — Heartbeats in, one light per board out

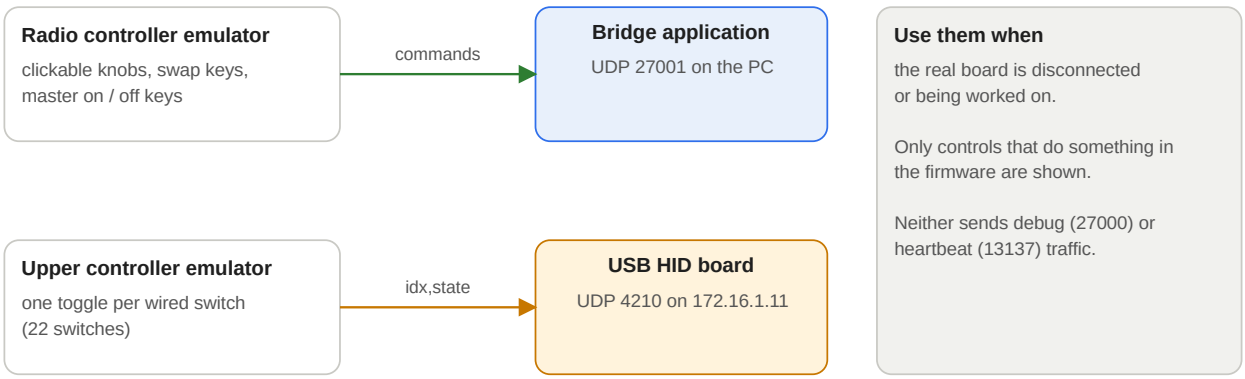
12.1 Program flow

1. `Form1_Load` starts a background task that binds a `UdpClient` to port **13137** and loops on `ReceiveAsync()`.
2. Each packet is matched by **prefix** (not by sender address): `COMM_NAV`, `SERVO`, `JOYSTICK`, `UPPER_INPUT`, `STEPPER` or `DUAL_STEPPER`. A match updates that board's last-received time and turns its label green; each match can also be logged (sender and timestamp) to a list box.
3. A timer calls `CheckHealth()` about once a second: 30 seconds or more since the last packet turns the indicator **red** (connection lost), 15 seconds or more **orange** (connection warning), otherwise it stays green.
4. *Toggle logs* shows or hides the log list box and resizes the form.

The monitor is a passive listener: it never sends anything and does not care which bridge is running. The Dual Stepper's prefix is deliberately distinct from `STEPPER` (the monitor uses `StartsWith`, so overlapping prefixes would light the wrong indicator). The OLED controller sends no heartbeat.

13 Radio and Upper controller emulators

Two small Windows Forms panels stand in for the physical boards when they are not connected. Each shows only the controls that actually do something in the firmware.



The Radio emulator still sends a bare frequency for every radio, so only COM1 tunes from it.

(The real board sends labelled COM2_STANDBY:, NAV1_STANDBY: and NAV2_STANDBY: values, which both bridges understand.)

Figure 24 — The emulators and where they send

13.1 JetRangerRadioControllerEmulator

Shows the two character-OLED displays (COM and NAV), a clickable knob for each of the eight encoders (click the right half to turn one detent clockwise, the left half for counter-clockwise), the COM1 and COM2 *XFER* swap keys and the *MASTER ON* and *MASTER OFF* keys. It sends the same payloads the board would send to the bridge on UDP 27001 (default host `172.16.1.10`):

- each knob sends the new standby frequency as a bare ASCII number such as `121.010`;
- the XFER keys swap the local active and standby values and send `COM1_RADIO_SWAP`, or `COM2_RADIO_SWAP` followed by `AVIONICS_MASTER_SET`;
- the MASTER keys send `MASTER_BATTERY_ON` with `ALTERNATOR_ON`, or the two `_OFF` commands.

Limits: the emulator still sends a bare frequency for every radio, so only COM1 tunes correctly from it (the real board sends `COM2_STANDBY:`, `NAV1_STANDBY:` and `NAV2_STANDBY:`). It has no NAV swap keys and sends no debug (27000) or heartbeat (13137) traffic. Its knob model is 5 kHz per minor detent, not the 25 kHz and 50 kHz channel spacing the real board now uses.

13.2 JetRangerUpperControllerEmulator

Shows one toggle button per switch the Upper firmware acts on (22 switches in groups: Cabin Heat, Air, Circuit Breakers, Instruments, Electrical, Lights; green means on) and sends `index,state` packets to the USB HID board (default `172.16.1.11:4210`) exactly as the real board would. Indexes 7 to 10, 18 to 21, 29 to 38 and 40 to 175 are wired in the matrix but drive nothing and are not shown.

14 StepperVSITester

A Windows Forms bench tool that sends real-unit values or raw step counts straight to the boards' UDP handlers without a simulator. It started as a vertical speed tester (hence the name) and now covers the whole Stepper and Dual Stepper boards plus the servo pitch and bank.

14.1 What it sends

Target	Controls
Stepper .105	`VSI`, `AGL`, `ITT`, `IAS`, `TQ`, `N1`, `FUEL` (real values, with sliders for VSI, AGL, ITT, IAS); `OILT`, `OILP`, `XMSNT`, `XMSNP` (also broadcast to .106)
Dual Stepper .106	`ALT`, `RPME`, `RPMR` (broadcast to both stepper boards), `FUELLOAD`, `ELECTRICALLOAD`, `ZULU` (hour and minute boxes), `TEMP` (text)
Servo .102	`PITCH`, `BANK` (degrees, one decimal) and `PITCHRAW`, `BANKRAW`
Raw step panel	A drop-down of raw codes (`IASRAW`, `VSIRAW`, `ALTRAW`, `OILTRAW`, `OILPRAW`, `XMSNTRAW`, `XMSNPRAW`, `ITTRAW`, `RPMERAW`, `RPMRRAW`, `N1RAW`, `FUELRAW`, `TQRAW`, `FUELLOADRAW`, `ELECTRICALLOADRAW`) sent to both stepper boards, with -1 step and +1 step buttons

Each row has a slider or text box, *Send* (or Enter) and *Zero*. Values are sent as typed, with no conversion; `RPME` and `RPMR` carry one decimal. Raw codes that a board does not have are silent no-ops there (for example `FUELLOADRAW` on the Stepper board).

Notes: the raw panel still lists `FLAPS`, `AOA`, `GFORCE`, `SPDMAX` and `N1RAW`; the first four no longer exist on any board, and `N1RAW` does nothing on the Dual Stepper. `RPME` and `RPMR` are no longer handled by the Stepper board, so only the Dual Stepper acts on them. Run it **instead of** a bridge, since both would otherwise fight over what drives the gauges.

15 SwingTester

A Windows Forms (.NET 10) bench tool that triggers each gauge's start-up swing over UDP, one gauge at a time or all of them in a fixed order. Build with `dotnet build -c Release` in its folder; the executable is `bin\Release\net10.0-windows\SwingTester.exe`.

15.1 How it works

It sends a control packet `C,<COMMAND>:1` to `<board IP>:13136`. The boards do not report completion, so after each command the app waits a per-row number of seconds (editable in the window, not saved between runs) before sending the next. A swing blocks the board's loop; too short a wait lets the next command queue in the board's UDP buffer and the order across different boards drifts.



Figure 25 — Swing ALL sequence

The default waits are the swing times measured on the real boards plus about one second of margin: Dual Stepper 5.0 to 5.2 s (oil and transmission gauges, altimeter) and 6.4 to 6.5 s (load and RPM gauges); Stepper 9.5 to 10.7 s; Servo about 2.1 s. A full pass takes about 220 seconds.

15.2 Window

- The three board IPs at the top (defaults `172.16.1.105`, `.106`, `.102`).
- **Swing ALL (in order)**, **Stop** (cancels the remaining waits; a swing already started still finishes), **OLEDs ON** and **OLEDs OFF**, and the check box *Light OLEDs during Swing ALL* (on by default).
- One row per gauge (18 rows on the left) with its board, a seconds box and a **Swing** button; the running row is highlighted gold. The 15 warning-lamp buttons are on the right.
- **OLEDs ON** sends `C,OLEDTEST:1` to the Dual Stepper and **OLEDs OFF** sends `C,OLEDTEST:0`. The **Temp OLED** box (default `21C`) with **Send TEMP** sends `D,TEMP:<text>`.
- **Reset boards...** opens a dialog with a tick box per board (Radio .101, Servo .102, Upper .103, Stepper .105, Dual Stepper .106, USB HID .11; all ticked by default) and sends `C,RESET:1` to each ticked board

(the HID board on port 4210). The Dual Stepper takes about a minute to restart and the HID board briefly drops its USB devices. The button is disabled while a run is active.

15.3 Warning lamps

The Servo board's lamps are driven by ordinary `D,<code>:1` and `D,<code>:0` packets to `172.16.1.102:13136`. There is a toggle per lamp (gold when on), **Lamps ON (in order)** (one lamp at a time, *on gap* default 1000 ms), **Lamps OFF (in order)** (*off gap* default 400 ms) and the check box *Light warning lamps during Swing ALL* (on by default). Turn-off order, also the button order: `SCF`, `GENF`, `FFLTR`, `BD`, `EC`, `BHOT`, `TOPW`, `EOUT`, `LOWF`, `FPMP`, `TRC`, `TC`, `BTMP`, `TOTW`, `RLOW`.

Swing ALL with the check box on first lights the 15 lamps one at a time (about 15 seconds), then runs the swings, then turns the lamps off one at a time (always at the end, also after Stop or an error). The Servo board only drives a lamp when the incoming value differs from the state it last remembered, so keep the bridge off while testing.

15.4 Keep-alive

Every sketch resets its gauges after 30 seconds with no packet. During a full run some boards get no command for 100 seconds or more, so while any run is active (Swing ALL, an individual swing or the lamp sequence) the app sends `C,KEEPALIVE:1` to all three boards every 10 seconds (and once at the start). The boards ignore the command name but count the packet as received data. The keep-alive stops when the run ends, so lamps or OLEDs left on manually still time out after 30 seconds.

16 Debug listener (UDP reflector)

Every board sends debug text to `172.16.1.10:27000`. A small Python reflector on the primary PC prints each line and can forward a copy to a second listener, so an analysis script can read the same stream. Only one program can own the port, so use the reflector's forwarding rather than starting a second listener. Keep the reflector running while boards are being flashed and restarted: the start-up lines (Ethernet Started, I2C scan, Setup Complete) are the quickest check that a flash worked.

Other helpers in `Jet Ranger Python Code/`: `Export_From_SimConnect.py` (a SimConnect exporter), `Test_Receiver.py` (a generic UDP JSON receiver) and `send_wol.py` (wake-on-LAN).

17 PCB notes

Notes from reading the KiCad projects in `Jet Ranger/PCBs` (schematics, PCB layouts and netlists). Matching boards to sketches is by name.

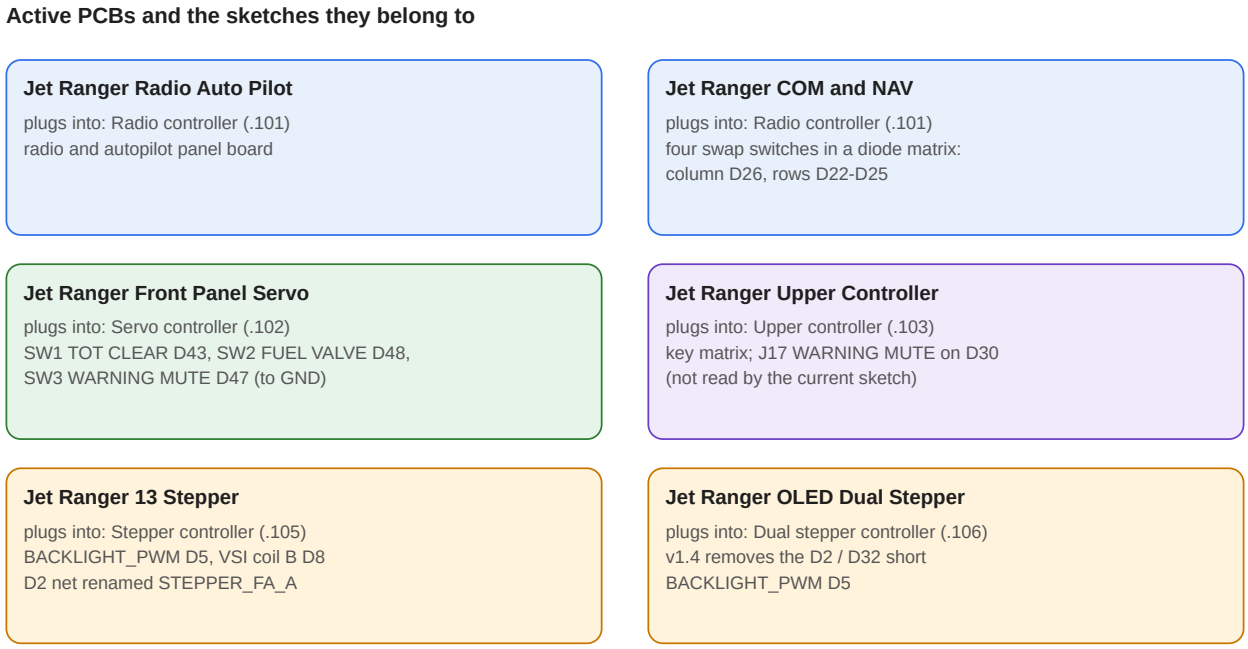


Figure 26 — Active PCBs and the sketches they belong to

17.1 Boards in use

PCB	Used by	Main parts
Jet Ranger Front Panel Servo	Servo controller	Mega shield, two ULN2803A lamp drivers, servo headers (16 on the PCB, 2 used), 15 annunciator outputs, three switch inputs
Jet Ranger 13 Stepper	Stepper controller	Mega shield, USR-ES1 Ethernet module, 14 six-pin gauge sockets, 12 V annunciator supply with polyfuses
Jet Ranger OLED Dual Stepper	Dual stepper controller	Mega shield, USR-ES1, six-pin gauge sockets, TCA9548A sockets for the OLEDs
Jet Ranger X27 Small Stepper	Each stepper gauge	One X27 or VID29 stepper, backlight LED, six-pin connector to the controller
Jet Ranger Radio Auto Pilot	Radio controller	Radio controller board
Jet Ranger COM and NAV	Radio controller	Front panel board: eight rotary encoders, push buttons and the four frequency-swap switches in a diode matrix
Jet Ranger Upper Controller	Upper controller	Mega shield for the switch matrix
Annunciator, Altimeter, Audio Panel, Auto Pilot Panel, ADF Panel, ADF Controller, Large Guage, Simkits gauge boards	Other panels and gauges	Not covered by the sketches in this document

The X27 steppers run straight from the Mega's pins, so the stepper boards need no driver chips; the 12 V input feeds annunciators and backlights.

17.2 Warning Mute, Fuel Valve and TOT Clear inputs

All KiCad schematic, PCB and netlist files under `Jet Ranger/PCBs` were searched for a Warning Mute (horn mute) input. Two boards have one.

- **Jet Ranger Front Panel Servo:** SW3 (`WARNING_MUTE`, a 2-pin Molex KK-254 connector) connects net `/SW_WARNING_MUTE` to Arduino Mega pin D47 and GND. The same board has SW1 `TOT_CLEAR` on D43 and SW2 `FUEL_VALVE` on D48, each also wired to GND. The Servo sketch reads these three pins as debounced inputs (internal pull-up, closed = LOW), logs every transition and sends it to the USB HID board.
- **Jet Ranger Upper Controller:** J17 (`WARNING MUTE`, a 2-pin Molex KK-254 connector) connects `/ROW8` (Arduino D30, shared with J18 TOT TEST) on pin 2 and, on pin 1, a 1N4148 diode D5 whose other side goes to `/COL0`, a switch in the board's diode-isolated key matrix. The Upper sketch scans a 4 by 3 matrix on analog pins A9 to A15, which does not include D30, so this input may not be read by the current firmware.

17.3 Pin and copper check (12 October 2026)

The KiCad boards were traced pad to pad through tracks and vias. No board had copper joining two controller pins in the final state, and every production board was fully routed. This checks connections only; it is not a design rule check (clearances, pours and footprints are not checked).

- **Jet Ranger OLED Dual Stepper:** up to version 1.3, D2 and D32 were one net (`STEPPER_FL_A`), joining fuel quantity and fuel load coil A. Fixed in version 1.4 (D2 is now `STEPPER_FA_A`). Boards older than 1.4 still have the short: fit only one of those two gauges or cut the trace. Make a fresh Gerber zip before ordering; the newest zip in `MANUFACTURING` was from 10 October.
- **Jet Ranger 13 Stepper:** D2 carried the same copied name but nothing else used it; it is renamed `STEPPER_FA_A`, label only, so no new board is needed.
- **Both stepper PCBs:** `BACKLIGHT_PWM` is D5 and VSI coil B is D8. Both stepper sketches use D5 for `BACK_LIGHTS` (pin 8 used to be both the backlight and VSI coil B).
- **Jet Ranger Front Panel Servo:** 10 of the 15 annunciator nets sit on different Mega pins from the sketch (A1/A2, A5/A6, A7 to A10, A11/A12). To be bench-tested lamp by lamp.
- **Upper Controller, 13 Stepper and OLED Dual Stepper:** pins otherwise match their sketches. The TCA9548A address pins (D17 to D19 on the Dual Stepper) are not driven; the module's pull-downs set address 0x70.

17.4 KiCad: changing the revision number

In the PCB Editor use *File, Page Settings*, set the *Title Block* revision field, then save the board. To show it on the board, place a text item on the `F.Silkscreen` layer (press T) containing `Rev ${REVISION}`; other variables such as `${TITLE}`, `${ISSUE_DATE}` and `${COMPANY}` work the same way. The schematic and the PCB each keep their own title block, so set both if they should match.

Appendix A: UDP token reference

From the current sketches. Gauge tokens take real units; lamp tokens take 0 (off) or 1 (on). `UDP_Interface_Values.xlsx` still lists the old 16-servo plan, so use these tables instead. Every gauge token also has a `RAW` twin (`PITCHRAW`, `IASRAW`) that takes a raw servo position or step count and skips the calibration table.

17.5 Gauge tokens

Token	Gauge	Board	Unit
`PITCH`	Attitude pitch	Servo .102	degrees, -30 to +30
`BANK`	Attitude bank	Servo .102	degrees, -90 to +90
`IAS`	Airspeed	Stepper .105	knots, one decimal allowed
`VSI`	Vertical speed	Stepper .105	ft/min
`AGL`	Radar altitude	Stepper .105	feet, 0 to 2500
`TQ`	Engine torque	Stepper .105	percent, 0 to 120
`N1`	Gas producer	Stepper .105	percent, 0 to 103
`ITT`	Turbine out temperature	Stepper .105 or Dual .106	degrees C
`FUEL`	Fuel quantity	Stepper .105 or Dual .106	US gallons
`OILT`	Engine oil temperature	Dual .106 (or Stepper)	degrees C
`OILP`	Engine oil pressure	Dual .106 (or Stepper)	psi
`XMSNT`	Transmission oil temperature	Dual .106 (or Stepper)	degrees C
`XMSNP`	Transmission oil pressure	Dual .106 (or Stepper)	psi
`RPME`	Engine speed (N2)	Dual .106	percent, one decimal
`RPMR`	Rotor speed	Dual .106	percent, one decimal
`ALT`	Altimeter	Dual .106, OLED .104	feet
`FUELLOAD`	Fuel load	Dual .106	psi
`ELECTRICALLOAD`	Electrical load	Dual .106	percent
`BARO`	Barometric setting	OLED .104	inHg x 100, for example `2992`

17.6 Display and status tokens

Token	Board	Meaning
`ZULU`	Dual .106	Zulu time as HHMM, for example `1430`
`TEMP`	Dual .106	Temperature text up to 8 characters, for example `25C`
`PWR`	Dual .106	1 = all four OLEDs on, 0 = blank (main bus below 24 V). Sent first in the packet
`C1A`, `C1S`, `C2A`, `C2S`	Radio .101	COM1 and COM2 active and standby frequency, MHz
`MAINBUS`	Radio .101	Main bus voltage
`NAVCOM1`	Radio .101	0 = avionics bus below 24 V (COM and NAV displays blank), 1 = on

17.7 Lamp tokens (Servo .102)

`EOUT`, `RLOW`, `TOPW`, `TOTW`, `BHOT`, `BTMP`, `EC`, `TRC`, `TC`, `BD`, `FFLTR`, `FPMP`, `GENF`, `LOWF`, `SCF` on pins A1 to A15 in that order (see the Servo controller chapter).

17.8 Control tokens (C,<COMMAND>:<value>)

Command	Meaning
`SWING<CODE>`	Run one gauge's start-up swing on demand, for example `C,SWINGTQ:1`; the value is ignored
`OLEDTEST`	1 fills the four Dual Stepper OLEDs solid white, 0 restores them
`KEEPALIVE`	Ignored command that still counts as received data and restarts the 30 second no-data timer
`RESET`	Reboot the board about 2 seconds later (port 13136; USB HID board on 4210)

Command	Meaning
MATRIX	Radio controller: print the raw keypad matrix to the debug stream
B	Panel brightness 0 to 15 (accepted, not yet active)

17.9 Messages from the Radio controller to the bridge (UDP 27001)

COM1_RADIO_SWAP, COM2_RADIO_SWAP, NAV1_RADIO_SWAP, NAV2_RADIO_SWAP, AVIONICS_MASTER_SET, MASTER_BATTERY_ON, MASTER_BATTERY_OFF, ALTERNATOR_ON, ALTERNATOR_OFF, COM2_STANDBY:<MHz>, NAV1_STANDBY:<MHz>, NAV2_STANDBY:<MHz> and a bare number for the COM1 standby frequency.

Appendix B: Building and flashing

17.10 Libraries

Ethernet 2.0.2 with SPI, AccelStepper 1.64.0 (steppers), Servo and Ewma (Servo controller), U8g2 and Wire (OLEDs), DCS-BIOS 0.3.13 (sketches that still carry DCS-BIOS code), and the Joystick and Keyboard libraries for the HID board. Install them from the Arduino Library Manager.

17.11 Compile and upload

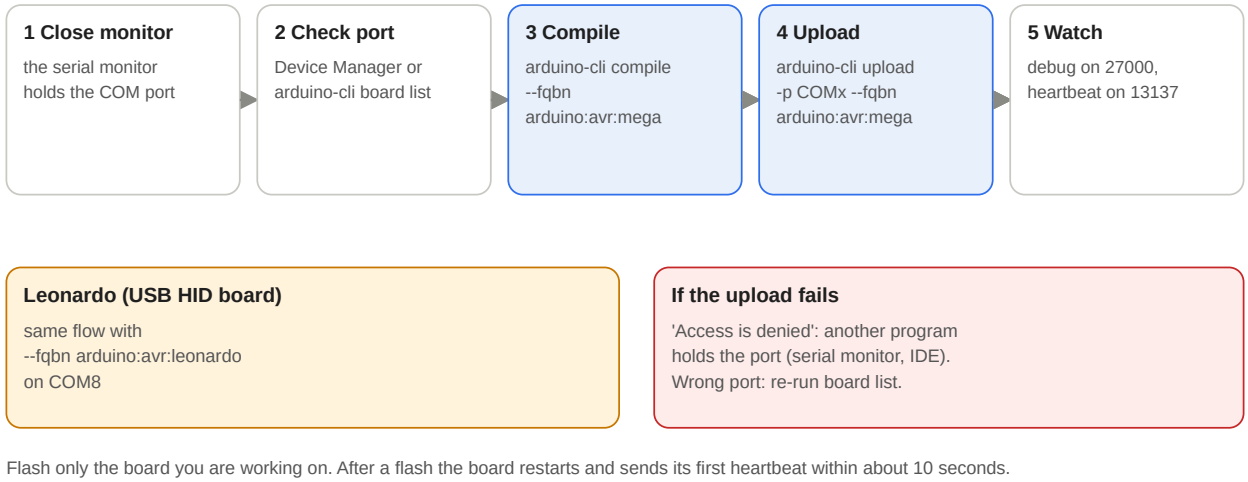
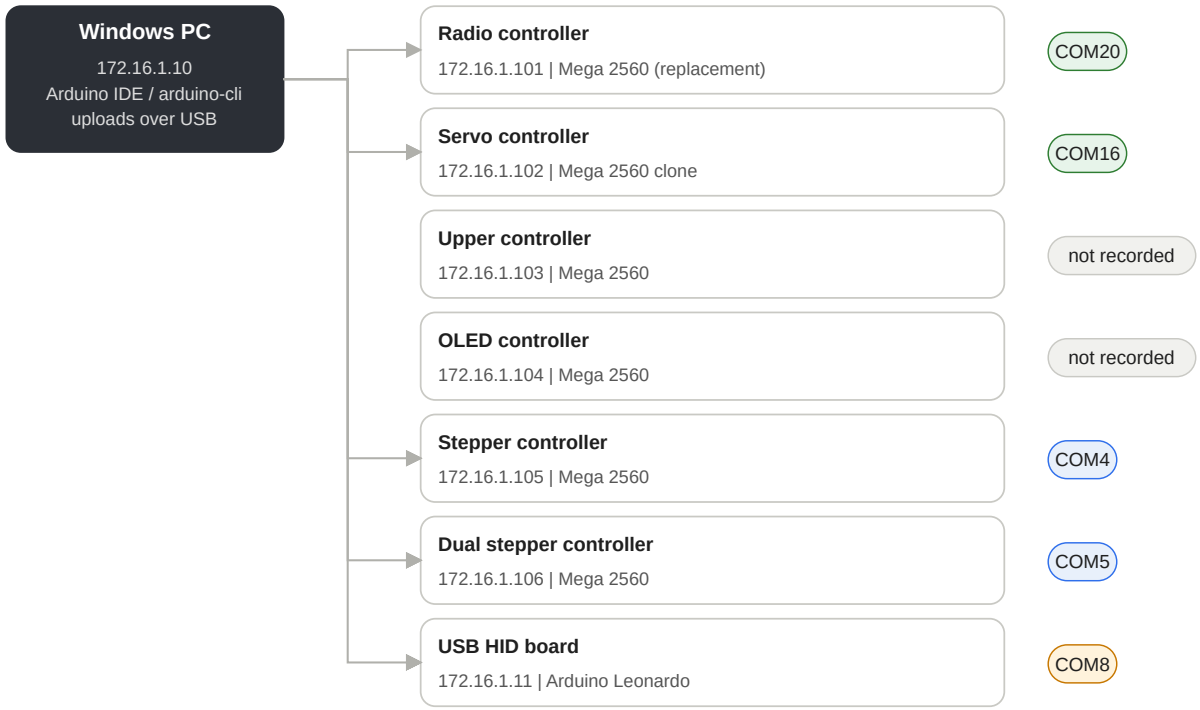


Figure 27 — Flashing a board

```
arduino-cli compile --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
arduino-cli upload -p COM4 --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
```

Use `--fqbn arduino:avr:leonardo` for the USB HID board. Close the Arduino IDE serial monitor first: it holds the port ("Access is denied"). Check the port with Device Manager or `arduino-cli board list`.

17.12 COM port to board mapping



COM numbers are the ones used at the last flash. They follow the USB socket and the board, so check them (see the guide) before an upload. Each board also has a network connection: USB is for programming only, the cockpit runs over UDP.

Figure 28 — Which board is on which COM port

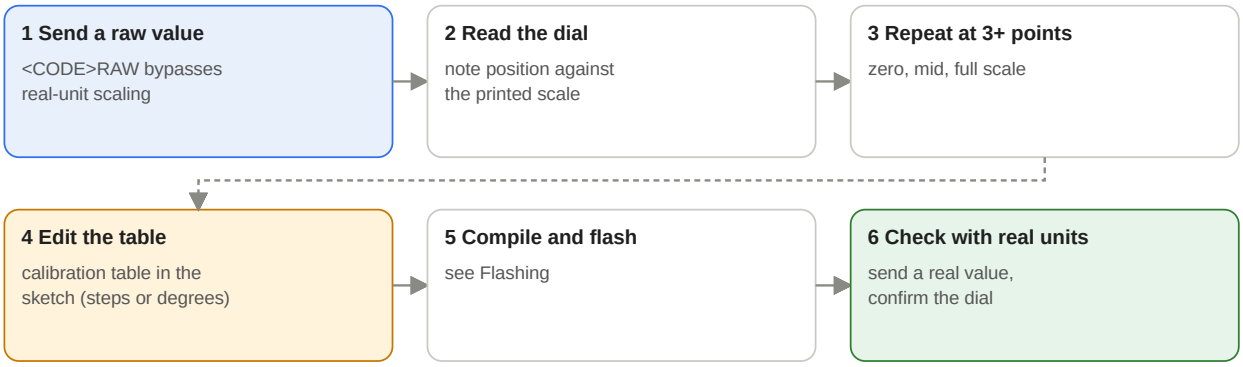
Board	IP	Hardware	COM port
Radio controller	.101	Mega 2560 (replacement board)	COM20
Servo controller	.102	Mega 2560 clone	COM16
Upper controller	.103	Mega 2560	not recorded
OLED controller	.104	Mega 2560	not recorded
Stepper controller	.105	Mega 2560	COM4
Dual stepper controller	.106	Mega 2560	COM5
USB HID board	.11	Arduino Leonardo	COM8

These are the ports used at the last flash of each board. They follow the USB socket and the board: the Servo clone has no USB serial number, so its COM number is tied to the socket it is plugged into, and a replaced Mega gets a new number. Check before every upload.

17.13 Sketch settings to know about

- Per-gauge `SwingXXX` gates and `SwingLoops` near the top of the Stepper and Dual Stepper sketches control which swings run at boot.
- Zero trims (`<CODE>ZeroOffset`, `TSoffset`, `RSoffset`) are set near the top of the same sketches.
- Every sketch has `reflectorIP` (172.16.1.10) for debug and heartbeat traffic.

Appendix C: Calibration



Typical tables: pitch -30 / 0 / +30 degrees, bank -90 / 0 / +90 degrees, one row per stepper or servo gauge.

Sending <CODE>RAW:n moves the motor to step n directly, so any dial can be calibrated without a simulator.

Figure 29 — The calibration loop

Between table points the boards interpolate in a straight line and hold the end value beyond the ends.

1. Send a **raw** value (`D,TQRAW:300`) to move the needle to a raw step or servo position.
2. Read the dial against its printed scale.
3. Repeat at three or more points (zero, mid scale, full scale).
4. Edit the calibration table in the sketch.
5. Compile and upload.
6. Send **real** values (`D,TQ:60`) and confirm the dial.

The StepperVSITester sends every raw and real token without a simulator. Do not drive a gauge hard against its stop: a stalled stepper or servo draws high current and gets hot. After any mechanical change to a gauge, recheck its table and zero trim.

Appendix D: How this document is built

This document and the User Guide are generated, not typeset by hand. The sources are Markdown files and vector diagrams in `Jet Ranger Documentation/DocBuild`:

Item	Purpose
`src/master/*.md`, `src/guide/*.md`	Text of this document and the User Guide
`diagrams_a.py` to `diagrams_d.py`, `svgdraw.py`, `make_diagrams.py`	The 30 diagrams, written as Python functions that draw SVG; `python make_diagrams.py` regenerates them
`build_docs.py`, `run_lo.ps1`	Builds the `.odt` and `.pdf` through LibreOffice's Python interface (`run_lo.ps1 build_docs.py master`)
`pdf2png.ps1`, `montage.ps1`, `svg2png.py`	Helpers for checking pages and diagrams visually

To change a diagram edit its function and rebuild; to change text edit the Markdown and rebuild. Keep `PROGRAM_SUMMARY.md` files and these sources in step after a code change.